



Estándar de Desarrollo de Aplicaciones Tecnológicas – Plataforma Jakarta EE

NAC-PTITARE26-00000016

***Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información -Datos
Dirección Nacional de Tecnología***

Septiembre 2026

Versión: 2.0.0

Historial de Versiones:

Versión	Fecha	Autor	Descripción
1.0.0	2015-11-19	Planificación TI	Emisión Inicial
1.1.0	2017-08-28	Dennys Lara	Modificación del Manejo de Logs
1.1.0	2018-06-15	Liliana Rodríguez	Actualización general (cambio de formato, ajustes en secciones de: pruebas unitarias, CDI, servicios web, beans, JPA, mensajería, logs y excepciones; reubicación y ajustes de catálogos y parámetros).
1.1.1	2020-05-12	José Díaz	Cambio sección 4.1.1 Configuración a nivel de Java EE 7 de manejos de logs, inclusión de logs para mecanismos.
1.2.0	2020-07-10	Christian Arreaga	Cambios en las secciones Estándares generales, Java Persistence API, Manejo de excepciones, Servicios Web, Anexos. Se agrega la sección Lógica Parametrizable.
1.2.1	2022-09-09	Carmen Muñoz	Cambios en la sección 2.2.2 Implementación de pruebas unitarias, se modifica la regla que menciona las clases que están excluidas de realizar pruebas unitarias.
2.0.0	2026-09	Benito Cuenca C.	Se añade regla de uso de SpotBugs, se agrega rutas de reglas definidas (PMD, Checkstyle, SpotBugs), actualización de documento de Patrones de Diseño Empresariales Jakarta EE, se añade regla general para implementar componentes de software, se añade reglas CDI, se añade reglas de componentes de negocio CDI, actualización de reglas de componentes EJB, se añade regla de Transacciones, se añade reglas de mensajería, se añade reglas de log.

Información del Documento

Resumen:	El presente documento establece los estándares, lineamientos y buenas prácticas técnicas la implementación de aplicaciones empresariales sobre la plataforma Jakarta EE. Su propósito es promover una arquitectura homogénea, mantenible, segura, escalable y de alto rendimiento mediante el uso consistente de las especificaciones y tecnologías soportadas por la plataforma, garantizando la calidad, interoperabilidad y sostenibilidad de los componentes de software desarrollados para la Institución.
Alcance:	Define las reglas, criterios técnicos y buenas prácticas que se deben cumplir al construir aplicaciones empresariales en la Institución basadas en la plataforma Jakarta EE. También se da lineamientos tales como: manejo de logs y excepciones, administración de catálogos y parámetros generales. Es menester decir que no se enseña a programar componentes empresariales sobre la plataforma Jakarta EE. El presente documento entra en vigencia a partir de la fecha de aprobación, y debe ser aplicado en la construcción de componentes de software que usen la Plataforma Jakarta EE.
Audiencia:	Dirección Nacional de Tecnología, Proveedores Externos

Cláusula de Responsabilidad

Los firmantes de este documento, en representación de sus unidades administrativa a las que pertenecen, aceptan y aprueban el presente documento y a su vez se comprometen a respaldar su cumplimiento, hacer difusión de este en sus respectivas unidades y a sugerir mejoras para versiones futuras en el caso de que se las identifiquen.

Firmas de Responsabilidad

Fecha de vigencia:	Dirección Nacional de Tecnología	Versión	Páginas
11/09/2026		2.0.0	70
Estándar de Desarrollo de Aplicaciones Tecnológicas – Plataforma Jakarta EE			
CARGO		FIRMA	
APROBADO POR:	Ing. Paulina Correa Aguayo DIRECTOR NACIONAL DE TECNOLOGÍA	 Validar únicamente en FirmaEC. Firmado electrónicamente por: PAULINA ELIZABETH CORREA AGUAYO	
REVISADO POR:	Ing. Johanna Chagñay Carpio JEFE NACIONAL DEPARTAMENTO PLANIFICACIÓN TI Y GESTIÓN INFORMACIÓN - DATOS	 Validar únicamente en FirmaEC. Firmado electrónicamente por: JOHANNA ESPERANZA CHAGNAY CARPIO	
	Ing. Alexandra Pasquel Pintado JEFE NACIONAL DEPARTAMENTO DESARROLLO	 Validar únicamente en FirmaEC. Firmado electrónicamente por: ALEXANDRA ELIZABETH PASQUEL PINTADO	
	Ing. Joffre Taco Rodríguez JEFE NACIONAL DEPARTAMENTO SEGURIDAD INFORMÁTICA	 Validar únicamente en FirmaEC. Firmado electrónicamente por: JOFFRE ALEXANDER TACO RODRIGUEZ	
	Ing. Patricio Cañarte Mantuano COORDINADOR NACIONAL DESARROLLO Y MANTENIMIENTO	 Validar únicamente en FirmaEC. Firmado electrónicamente por: ROBERTO PATRICIO CANARTE MANTUANO	
	Ing. Susana Samaniego Freire COORDINADOR NACIONAL CONTROL Y ASEGURAMIENTO DE CALIDAD	 Validar únicamente en FirmaEC. Firmado electrónicamente por: SUSANA ALEXANDRA SAMANIEGO FREIRE	
	Ing. Edison Cayambe Chicaiza COORDINADOR NACIONAL GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA	 Validar únicamente en FirmaEC. Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA	
ELABORADO POR:	Ing. Benito Cuenca C. EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS	MANUEL BENITO CUENCA CUENCA  Digitally signed by MANUEL BENITO CUENCA CUENCA Date: 2026.08.26 10:06:14 -05'00'	

Enmienda del Documento

El Departamento de Planificación TI y Gestión de Información - Datos tiene la potestad de actualizar el contenido de los Anexos cuando lo considere necesario y no requerirá recolección de firmas para su aprobación. Sin embargo, se comunicarán a las Coordinaciones y Departamentos involucrados los cambios a realizarse, quienes podrán indicar sus observaciones al respecto en los siguientes ocho días laborables, luego de lo cual se considerarán aceptadas, y se procederá a su publicación.

Ante cualquier sugerencia de mejora al documento debe ser comunicado al Departamento de Planificación TI y Gestión de Información – Datos para su análisis, evaluación de factibilidad y posterior actualización de la documentación en caso de considerarse necesario.

En caso de ser necesario emitir un lineamiento técnico, aclaración o alcance al presente documento, este podrá ser emitido mediante memorando suscrito por el jefe Departamental de Planificación TI y Gestión de Información – Datos en coordinación con la Dirección Nacional de Tecnología; y se mantendrá como una disposición vigente hasta su incorporación en la siguiente actualización del documento.

Tabla de contenidos

1	Introducción	9
2	Estándares Generales.....	9
2.1	Convención de codificación para Java SE	11
2.2	Pruebas unitarias.....	11
2.2.1	Desarrollo guiado por pruebas (TDD).....	11
2.2.2	Implementación de pruebas unitarias	12
2.3	Patrones de Diseño Empresariales Jakarta EE	13
2.4	Catálogos y parámetros	13
2.5	Interoperabilidad.....	13
2.6	Escalabilidad	15
3	Componentes Plataforma Jakarta EE	15
3.1	CDI	15
3.1.1	Empaquetamiento de aplicaciones CDI.....	16
3.1.2	Métodos producir, disponer y atributos producir en aplicaciones CDI	16
3.1.3	Beans predefinidos en aplicaciones CDI.....	17
3.1.4	Eventos en aplicaciones CDI	17
3.1.5	Interceptores en aplicaciones CDI.....	18
3.1.6	Decoradores en aplicaciones CDI	19
3.1.7	Estereotipos en aplicaciones CDI.....	19
3.2	Componentes de Negocio	20
3.2.1	Servicios CDI	21
3.2.2	Session Beans	22
3.2.3	Message-Driven Bean (MDB)	23
3.2.4	Transacciones	23
3.2.5	Rollback de transacciones gestionadas por contenedor	25
3.3	Java Persistencia API - JPA.....	26
3.3.1	Pool de conexiones.....	28
3.3.2	Unidad de persistencia	28
3.3.3	Java Persistence Query Language (JPQL)	29
3.3.4	Critería API.....	30
3.3.5	Ejecución de procedimientos almacenados	31

3.3.6	Entidades	32
3.3.7	Clases embebible en entidades	33
3.3.8	Mapeo de relaciones entre entidades.....	34
3.3.9	Caché de segundo nivel	37
3.4	Validaciones de los Java Bean	37
3.4.1	Constraints de Bean Validation	38
3.4.2	Validación de métodos en CDI Managed Beans.....	40
3.4.3	Creación de constraints personalizados.....	41
3.4.4	Validación de servicios web REST.....	43
3.4.5	Mapeo de excepciones de Bean Validation con JAX-RS.....	43
3.4.6	Personalización de mensajes de constraints de Bean Validation	44
3.5	Mensajería.....	45
3.5.1	Reglas de uso.....	45
3.5.2	Producir y consumir.....	45
3.5.3	Procesamiento por lotes	46
4	Manejo de logs y excepciones.....	46
4.1	Manejo de logs	46
4.1.1	Configuración a nivel de Jakarta EE.....	46
4.1.2	Implementación general del log.....	47
4.1.3	Implementación de mensajes logs	49
4.1.4	Implementación del log con MDC	50
4.1.5	Niveles de un log	50
4.1.6	Correlación de la información del log con las pistas de auditoría.....	51
4.1.7	Herramienta de análisis de logs	51
4.2	Manejo de Excepciones.....	51
4.2.1	Implementación de excepciones:.....	51
4.2.2	Excepciones en el contexto de componentes de negocio CDI	54
4.2.3	Excepciones en el contexto EJB.....	54
4.2.4	Especificaciones de la Implementación.....	56
4.2.5	Excepciones en el Contexto WEB: Servlet, JSF, JAX-WS (SOAP), JAX-RS (REST)....	56
5	Lógica Parametrizable	57
6	Servicios Web	58
6.1	Servicios Web (SOAP – JAX-WS).....	58

6.2	Servicios Web REST (JAX-RS)	59
6.2.1	Configuración	59
6.2.2	Implementación	60
6.2.3	Validaciones con JavaBeans	62
6.2.4	Respuestas.....	63
7	Glosario	64
7.1	Acrónimos	64
7.2	Definiciones.....	64
8	Bibliografía	66
9	Anexos.....	67
9.1	Anexo 1: Estándares soportadas por la Plataforma Jakarta EE.....	67
9.2	Anexo 2: Tecnologías/Herramientas a utilizar	69
9.3	Anexo 3: Recursos relacionados.....	69

1 Introducción

En el Servicio de Rentas Internas para el desarrollo de aplicaciones tecnológicas empresariales, la tecnología estándar es Java con la Plataforma Jakarta EE. El presente documento establece las definiciones técnicas obligatorias para el desarrollo de aplicaciones basadas en la plataforma Jakarta EE, con el fin de garantizar uniformidad, calidad, seguridad y mantenibilidad en las soluciones tecnológicas institucionales. Así como también, definir el uso adecuado de componentes, prácticas y arquitecturas estándar, la interoperabilidad entre sistemas, optimizar el rendimiento y asegurar la escalabilidad de las aplicaciones, alineando el desarrollo de aplicaciones con las mejores prácticas actuales de la industria.

2 Estándares Generales

Regla: Los estándares definidos en este documento son de cumplimiento obligatorio en el desarrollo de aplicaciones institucionales. Para casos de excepción no descritos en este documento se deberá consultar y gestionar la aprobación a la Coordinación de Gestión de Datos y Arquitectura Tecnológica previo cualquier desarrollo. En caso de generarse una excepción se documentará con las debidas justificaciones del requirente y aprobadas por la Coordinación de Gestión de Datos y Arquitectura Tecnológica.

Regla: Todo proyecto de software nuevo debe utilizar las tecnologías y versiones de la Plataforma Jakarta EE soportada en el servidor de aplicaciones. Como referencia ver Anexo 1 "[Estándares soportadas por la Plataforma Jakarta EE](#)".

Regla: Todo proyecto de software nuevo debe utilizar el arquetipo Jakarta EE del SRI para el desarrollo de aplicaciones tecnológicas institucionales, para lo cual, referirse al readme.md del proyecto base correspondiente. Ver ruta de proyectos base en Anexo 3: [Recursos relacionados](#).

Utilidad: El uso del arquetipo garantiza que el proyecto sea construido con MAVEN, utilice la última versión de JDK soportada por el servidor de aplicaciones, estructura definida de la aplicación y look & feel.

Regla: Es obligatorio el uso de PMD con las reglas definidas. Ver ruta de reglas en Anexo 3: [Recursos relacionados](#).

Utilidad: PMD ayuda a detectar código ineficiente o malos hábitos de programación.

Regla: Es obligatorio el uso de Checkstyle con las reglas definidas, ver ruta de reglas en Anexo 3: [Recursos relacionados](#).

Regla: Es obligatorio el uso de SpotBugs con las reglas definidas, ver ruta de reglas en Anexo 3: [Recursos relacionados](#).

Regla: Excluir de las validaciones de PMD, CheckStyle y SpotBugs las clases generadas por herramientas de terceros como, por ejemplo:

- ✓ Clases metamodelo basadas en la especificación JPA que tienen el mismo nombre de la clase administrada en la unidad de persistencia más un underscore "_" al final de su nombre, por ejemplo, la clase de persistencia Autor.java su metamodelo es Autor_.java. Del mismo modo se deben excluir de dicha validación los siguientes archivos creados en tiempo de compilación:

```
**/*_*.java  
**/*_$.bundle.java  
**/*_$.logger.java
```

✓ Clientes de servicios SOAP generados a partir de la herramienta wsimport incluida en el JDK.

Regla: En el código fuente de las aplicaciones NO se deben fijar los valores de: direcciones IP, nombres de dominio, URLs, puertos, protocolos, nombre jndi, nombre de servidor, correo electrónico, usuarios, claves o cualquier otro valor que sea identificado como potencialmente variable en el tiempo de vida de la aplicación, estos valores deben ser manejados mediante parametrización; para el caso de claves utilizar el repositorio de credenciales del servidor de aplicaciones.

Regla: Cumplir los lineamientos establecidos para cada base de datos que existen en la institución, estos lineamientos son de cumplimiento obligatorio y se encuentran descritos en el documento Estándar Tecnológico de Objetos de Base de Datos.

Regla: Los componentes transaccionales que requieran acceder a la base de datos, deben realizar las conexiones según su ambiente (Internet e Intranet). Es decir, si el componente corresponde a funcionalidad que se ejecuta en la intranet solo podrá conectarse a las bases de datos de ese ambiente (ejemplo: INTRASRI, SRI) y si el componente es de internet solo podrá conectarse a las bases de internet (ejemplo: INTER, INTERSRI). Para casos de excepción a esta regla se deberá contar por cada componente con la aprobación del arquitecto asignado al proyecto y la revisión del Departamento de Seguridad Informática.

Utilidad: Manejar el bajo acoplamiento en las aplicaciones y la independencia entre ambientes.

Regla: Para la implementación de procesamiento batch se deberá aplicar lo establecido en el documento “*Estándar de Desarrollo de Aplicaciones Batch*”. Ver ruta en [Anexo 3: Recursos relacionados](#).

Regla: En la invocación desde un cliente (navegador) a cualquier URL web tales como servicios web (REST / SOAP), páginas web desarrolladas en cualquier tecnología, servlets u otra lógica a nivel del servidor, no se deben pasar como parámetros cualquier información que se pueda obtener mediante otro mecanismo, como por ejemplo de las cabeceras de la petición http, de información de sesión y/o token o que pudiera ser consultada en los repositorios institucionales en base a los datos anteriores.

Nota como ejemplo:

No se debe enviar como parámetro los datos de:

- IPs: en su lugar se debe obtener del request.
- Número de identificación, nombre o correo del usuario: en su lugar se debe consultar en la base de datos con la identificación de la sesión.

Utilidad: Reducir riesgos de seguridad como, por ejemplo, que un contribuyente pueda obtener información de otro.

Regla: En el caso de que se requiera un servicio web para obtener información que constituyen datos privados (que le pertenece al exclusivamente usuario autenticado), se debe validar que las consultas retornen únicamente datos de ese usuario.

Utilidad: Reducir riesgos de seguridad como, por ejemplo, que un contribuyente pueda obtener información de otro.

Ejemplo: Cuando un usuario ingresa al sistema y consulta una de sus declaraciones, se debe asegurar que dicha consulta no solo se realice la búsqueda en base al código de la declaración, sino que también filtre por la identificación de la persona que está autenticada en sesión.

Regla: Todas las aplicaciones tecnológicas deben estar integradas al Single Sign On institucional. Para conocer la ubicación de la guía de administración revise [Anexo 3: Recursos relacionados](#).

Regla: Para el uso de librerías de terceros por fuera de las propias del estándar Jakarta EE y de aquellas que se encuentran por defecto en los módulos del servidor de aplicaciones referir a la sección “Librerías Externas” del documento “Estándar de Configuración de Servidores de Aplicaciones JBoss EAP 8.x”.

Regla: Los componentes de software deben implementarse cumpliendo con los principios arquitectónicos descritos en el documento “Arquitectura Tecnológica de Referencia Software” y a los principios de código limpio descritos en el documento “Estándar Tecnológico Codificación Lenguaje de Programación Java SE”.

2.1 Convención de codificación para Java SE

Se definen estándares o convenciones de codificación porque un estilo de programación homogéneo en un proyecto permite que todos los participantes lo puedan entender en menos tiempo y que el código en consecuencia sea mantenible.

El detalle referente a este tema se lo puede ver en el documento llamado “Estándar Tecnológico *Codificación Lenguaje de Programación Java SE*”. Ver ruta en [Anexo 3: Recursos relacionados](#).

2.2 Pruebas unitarias

2.2.1 Desarrollo guiado por pruebas (TDD)

Regla: Para la implementación de un nuevo proyecto se debe aplicar la técnica de desarrollo guiado por pruebas, en inglés Test Driven Development (TDD).

Utilidad: Con el desarrollo guiado por pruebas se favorece la construcción de aplicaciones de mayor calidad; se enfoca a las necesidades del cliente facilitando el entendimiento del problema, y disminuyendo el tiempo de depuración de errores.

La siguiente imagen muestra el ciclo de desarrollo de pruebas unitarias con TDD:

CICLO DE DESARROLLO DE PRUEBAS UNITARIAS CON TDD (Test-Driven Development)



Ciclo de desarrollo de TDD

2.2.2 Implementación de pruebas unitarias

Regla: Para implementar las pruebas unitarias se debe tomar en cuenta los siguientes puntos:

- Los frameworks o librerías que se deben utilizar para hacer las validaciones o afirmaciones de los casos de prueba unitarias es '**JUnit**' y/o '**hamcrest**'. Hamcrest permite escribir validaciones de forma más clara y fácil, además los errores en los 'assert' son más descriptivos que junit.
- Se debe utilizar el framework **Mockito** para la implementación de pruebas unitarias de componentes que requieran simulación (mocking) de dependencias, tales como servicios CDI, EJB, REST, etc.
- No se requiere hacer pruebas unitarias explícitas de: las entidades de persistencia (Entity), java beans (getter y setter), POJO's ya sean estos usados como DTO (Data Transfer Object) o como TO (Transfer Object), DAO (Data Access Object), clases usadas para definir excepciones, definición de interfaces, enumeraciones, clases usadas para las pruebas unitarias, clases utilizadas exclusivamente para constantes, static Data Model, clases con anotación @Produces (*Métodos producir, disponer y atributos producir en aplicaciones CDI*), cumpliendo el porcentaje de cobertura.
- El porcentaje de cobertura del código debe respetar lo definido en el Procedimiento de Desarrollo y Mantenimiento de Aplicativos Informáticos Institucionales, sección 3.1.8 "LA CALIDAD" DURANTE EL PROCESO DE DESARROLLO, literal "g".
- Se debe indicar un mensaje descriptivo en todas las validaciones o afirmaciones ('asserts').
- Las pruebas deben ser independientes unas de otras.

- En las pruebas unitarias, no se debe dejar una cadena de caracteres en un fichero de log, es decir no se debe registrar logs de las pruebas para luego ser revisadas su éxito o falla.

2.3 Patrones de Diseño Empresariales Jakarta EE

El detalle referente a este tema se lo puede ver en el documento llamado “*Guía Referencial de Patrones de Diseño Empresariales – Plataforma Jakarta EE*” (Ver ruta en Anexo 3: Recursos relacionados).

2.4 Catálogos y parámetros

Reglas:

- La estrategia que se debe utilizar para el registro, acceso, actualización y desactivación de parámetros transversales y específicos, es a nivel de base de datos y se los debe acceder mediante la funcionalidad de parametro-ambiente (sri-adm-catalogo-generico).
- Todo catálogo transversal debe estar centralizado en el módulo de ADM (Administración). Para acceder a dicha información se debe utilizar servicios genéricos ofrecidos por dicho módulo.
- Se debe implementar servicios genéricos para obtener los registros de los diferentes tipos de catálogos. Estos servicios deben poder ser *reutilizados* por los tipos de aplicaciones (empresariales, de escritorio, APIs) de la Institución que requieran acceder a los registros de los catálogos.
- Para el ambiente de Contingencia se utilizarán los parámetros correspondientes a este ambiente, configurados en la infraestructura tecnológica.
- Los registros de parámetros y catálogos deben utilizar mecanismos de caché cuando sean consultados frecuentemente y presenten baja frecuencia de modificación. La carga del caché podrá realizarse por demanda (lazy loading) o mediante precarga (eager loading) de acuerdo con el tipo de dato a almacenar. [Ver Caché de segundo nivel].

2.5 Interoperabilidad

Regla: El mecanismo por defecto para la interoperar entre sistemas transaccionales es el uso de componentes genéricos que implementan funcionalidades comunes entre los distintos sistemas. Estos componentes deben limitarse a funcionalidades transversales, no incluir clases innecesarias y deben cumplir con la regla.

Utilidad: Facilita el reusó de funcionalidades y la interoperabilidad entre sistemas transaccionales manteniendo un mismo contexto transaccional. Utilizar mejores prácticas en la implementación de componentes de software.

Regla: El despliegue de artefactos genéricos debe realizarse de acuerdo con el ambiente al que pertenece la funcionalidad que lo va a utilizar (Internet o Intranet).

Utilidad: El usar genéricos por contexto (Internet o Intranet) permite una correcta segregación de funcionalidades/operaciones lo que ayuda a tener una cohesión alta de componentes, es decir, en el ambiente de Internet solo se puede hacer uso de genéricos de Internet conectados

a las bases de Internet, de la misma manera en el ambiente de Intranet solo se puede hacer uso de genéricos de Intranet conectados a las bases de Intranet.

Regla: Para la interoperabilidad mediante API Restful se debe tomar en cuenta las siguientes consideraciones:

- ✓ Esta integración puede utilizarse en los siguientes casos:
 - Para integraciones entre aplicaciones en las cuales no se puede utilizar artefactos genéricos.
 - Para desarrollo de lógica backend que se consumirá desde aplicaciones desarrolladas con frameworks Javascript, tales como Angular.
 - Para interactuar con proyectos Batch.
- ✓ Los servicios RESTful de interoperabilidad deben cumplir con las reglas dispuestas en la sección [5 Servicios Web](#).
- ✓ El consumo seguro de API's Restful desde código PLSQL debe realizarse usando autenticación mediante credenciales que se almacenan en Oracle Wallet.

Regla: La interoperabilidad mediante Java Message Service aplica bajo las siguientes consideraciones:

- ✓ Cuando la funcionalidad del sistema implica la recepción de miles solicitudes por segundo y su procesamiento puede realizarse de manera asíncrona.
- ✓ Cuando las solicitudes pudieran generar procesamientos paralelos de lógicas que dependen del mismo evento de la creación de la solicitud.
- ✓ Cuando exista la necesidad de reintentos de la solicitud de procesamiento por una eventual falla.
- ✓ Se debe utilizar la implementación y el motor JMS que esté disponible por defecto en el servidor de aplicaciones. (Ver más en [3.5 Mensajería](#))

Ejemplo:

Procesamiento y envío masivo de solicitudes de emails

Regla: La interoperabilidad mediante base de datos puede darse en los siguientes escenarios:

- ✓ Consumo de información entre esquemas que pueden pertenecer o no a un mismo ambiente (Intranet o Internet) para lo cual se puede usar réplicas mediante vistas materializadas o permisos directos en función de lo analizado y aprobado por el Arquitecto TI asignado.
- ✓ Consumo de información de entidades externas donde el mecanismo definido para compartir información es la réplica entre bases de datos.
- ✓ Ejecución de lógica legada desarrollada en PL/SQL mediante el uso de paquetes de servicios.

Nota: Para revisar las reglas de los objetos de base de datos revise “Estándar Tecnológico de Objetos de Base de Datos”. (Ver [Anexo 3: Recursos relacionados](#))

Regla: La interoperabilidad con entidades externas se debe realizar mediante servicios web de tipo SOAP según lo dispuesto en la RESOLUCION No N-44-NG-DINARDAP-2016-ANEXO1 (Ver [Anexo 3: Recursos relacionados](#)). Para verificar las reglas de los servicios SOAP revisar [5 Servicios Web](#)

2.6 Escalabilidad

Regla: Las aplicaciones deben tener un diseño que permita garantizar la escalabilidad horizontal, para lo cual se debe cumplir lo siguiente:

- Las aplicaciones deben tener datasources que usen Oracle RAC, siempre que se encuentre disponible para el ambiente en el que se despliegan.
- Los modelos relacionales y la lógica de los aplicativos deben permitir escalamiento horizontal, es decir, la transaccionalidad de un aplicativo pueda ser gestionada por múltiples instancias de servidores de aplicaciones.
- Las aplicaciones cuyas funcionalidades necesiten sincronización en sesiones, subsistemas de memoria de cache de segundo nivel, timers, etc. Deben desplegarse en servidores de aplicación en configuración High Availability.

3 Componentes Plataforma Jakarta EE

La plataforma Jakarta EE proporciona un conjunto de especificaciones para desarrollar aplicaciones empresariales mediante componentes administrados por el contenedor, los cuales pueden implementarse según las necesidades funcionales y no funcionales de la aplicación.

De manera general, las aplicaciones institucionales podrán estar conformadas por:

Componentes transaccionales, orientados a implementar la lógica de negocio y el procesamiento de solicitudes en línea, utilizando tecnologías como CDI, Jakarta Transactions, Jakarta Persistence, Jakarta REST, Jakarta XML Web Services y Enterprise Beans cuando se requieran capacidades específicas.

Componentes de procesamiento por lotes (Batch), orientados a la ejecución de procesos masivos o programados, utilizando Jakarta Batch u otros mecanismos administrados por la plataforma, según la arquitectura de la solución.

Tanto los componentes transaccionales como los de procesamiento por lotes podrán utilizar de forma común las especificaciones de la plataforma Jakarta EE de acuerdo con las necesidades de cada componente.¹

3.1 CDI

Regla: Para la inyección de dependencia de objetos de componentes de negocio CDI (*@Transactional*) y de objetos de componentes de Enterprise Beans (EJB) local se debe usar por defecto la anotación **@Inject** (paquete jakarta.inject.Inject).

Utilidad: Unifica el mecanismo de inyección de dependencias, reduce el acoplamiento con Enterprise Beans y favorece el modelo de programación basado en CDI recomendado por Jakarta EE, manteniendo el acceso a las capacidades avanzadas de EJB cuando sean necesarias.

¹Para mayores detalles respecto a los temas tratados en esta sección, referirse a la documentación oficial <https://jakarta.ee/learn/docs/jakartaee-tutorial/current/index.html> de la plataforma Jakarta EE.

Regla: El uso de la anotación `@EJB(lookup="...")` debe limitarse a escenarios de consumo remoto de EJB entre módulos independientes (WAR, JAR, EAR).

Utilidad: Garantiza resolución exacta del EJB dentro del módulo independiente. Reduce acoplamiento a JNDI y mejora mantenibilidad y portabilidad de la aplicación.

Regla: La inyección de dependencias de los beans administrados por CDI debe realizarse mediante la anotación **@Inject** (paquete `jakarta.inject.Inject`).

Utilidad: Garantiza que la inyección de dependencias sea administrada por el contenedor CDI, promoviendo el desacoplamiento entre componentes, la reutilización y el cumplimiento del modelo de programación de Jakarta EE.

3.1.1 Empaquetamiento de aplicaciones CDI

Regla: En aplicaciones Jakarta EE, CDI debe utilizar el mecanismo de descubrimiento implícito de beans administrado por el contenedor. El archivo `beans.xml` solo deberá incluirse cuando sea necesario configurar explícitamente interceptores, decoradores, alternativas, especializaciones u opciones avanzadas de descubrimiento CDI. En los demás casos, no se debe incorporar el archivo `beans.xml`.

Utilidad: Simplifica la configuración de aplicaciones Jakarta EE, reduce artefactos innecesarios y favorece el uso del modelo moderno de descubrimiento automático de beans CDI definido por la especificación Jakarta CDI.

Nota: De ser necesario, `beans.xml` debe estar ubicado de acuerdo con el tipo de aplicación:

- Para una aplicación desplegada como `.war`, el deployment descriptor `beans.xml`, debe estar en el directorio **WEB-INF**.
- Para una aplicación desplegada como EJB o archivo JAR, el deployment descriptor `beans.xml`, debe estar en el directorio **META-INF**. Si en la aplicación existen varios `.jars`, para activar CDI debe existir un `META-INF/beans.xml` por `.jar`

3.1.2 Métodos producir, disponer y atributos producir en aplicaciones CDI

Regla: Se deben utilizar métodos o atributos `@Produces` (`jakarta.enterprise.inject.Produces`) de CDI cuando sea necesario controlar explícitamente la creación, configuración o inicialización de objetos cuya instanciación requiera lógica personalizada, parámetros de configuración, integración con recursos externos o creación distinta al constructor por defecto.

Utilidad: Permite centralizar y desacoplar la lógica de instanciación de dependencias complejas, facilitando integración con recursos administrados, configuración dinámica, reutilización de componentes y alineación con el modelo de inyección de dependencias de Jakarta EE.

Regla: Se deben utilizar métodos `@Disposes` de CDI para liberar, cerrar o finalizar recursos administrados cuya creación haya sido realizada mediante métodos `@Produces`, especialmente en componentes que manejen conexiones, recursos externos, clientes de integración, colas, sesiones, streams u objetos que requieran lógica explícita de liberación.

Utilidad: Permite administrar adecuadamente el ciclo de vida de recursos complejos, garantizando liberación controlada de recursos, reducción de fugas de memoria y alineación con el modelo moderno de gestión de dependencias y recursos de Jakarta EE.

3.1.3 Beans predefinidos en aplicaciones CDI

Regla: Jakarta EE provee beans predefinidos que deben ser inyectados en algún bean que lo requiera de acuerdo con sus utilidades expuestas en la siguiente tabla:

Tipo Bean	Descripción	Lugar de inyección
<code>jakarta.transaction.UserTransaction</code>	Transacción manejada por el bean	Session beans (BMT) o componentes webs (servlets, filtros y web services)
<code>java.security.Principal</code>	Obtiene el nombre o id del login de autenticación	Componentes webs (managed beans, servlets, filtros, listener o web service)
<code>jakarta.validation.ValidatorFactory</code> <code>jakarta.validation.Validator</code>	Validación completa o solo de un atributo de un JavaBean	Los JavaBeans creados deben ser desplegados en un contenedor web o EJBs.
<code>jakarta.servlet.http.HttpServletRequest</code>	Obtención de todos los detalles de la petición (request) de un cliente	Componentes webs (managed beans, servlets, filtros, listener y web service)
<code>jakarta.servlet.http.HttpSession</code>	Obtención información acerca de una sesión y objetos atados a una sesión	Componentes webs (managed beans, servlets, filtros, listener y web service).
<code>jakarta.servlet.ServletContext</code>	Comunicación con el servlet container mediante métodos (obtener el MIME type de un archivo, etc.) propios de este bean.	Componentes webs (managed beans, servlets, filtros, listener y web service).

3.1.4 Eventos en aplicaciones CDI

Los eventos CDI permiten implementar arquitectura orientada a eventos mediante publicación y observación de eventos síncronos o asíncronos (`@Observes`, `@ObservesAsync`), sin dependencias directas en tiempo de compilación entre los componentes participantes. A diferencia del EJB Timer Service, los eventos CDI no están orientados a planificación tipo cron, sino a notificación y propagación de eventos de negocio dentro de la aplicación.²

Regla: Los eventos CDI deben utilizarse para implementar comunicación desacoplada entre componentes relacionados, tales como: publicación de eventos de negocio, notificaciones internas o mecanismos de extensión de comportamiento. No deben utilizarse como reemplazo de invocaciones directas en flujos de negocio simples o secuenciales.

Utilidad: Permite implementar arquitecturas desacopladas y orientadas a eventos, favoreciendo extensibilidad, mantenibilidad y separación de responsabilidades en aplicaciones Jakarta EE.

Procedimiento:

1. Definición del evento

Un evento consiste en lo siguiente:

- El objeto evento que es un objeto java.

² https://jakarta.ee/specifications/cdi/4.0/jakarta-cdi-spec-4.0.html?utm_source=#events

- Cero o más tipos de cualificadores.

2. Control del evento

Un controlador de eventos usa métodos `observer` para consumir eventos. Cada método `observer` toma como parámetro un evento de un específico tipo de evento que es anotado con `@Observes` (paquete `jakarta.enterprise.event.Observes`) y si hay varios métodos `observer` para el mismo tipo de evento, se debe crear un cualificador para cada tipo de evento.

3. Disparo del evento

Para activar un evento se debe llamar al método `fire` de `jakarta.enterprise.event.Event`, este método dispara un evento y notifica a los métodos `observer`.

Nota: Los eventos son solo síncronos, para solucionar esto, si se usa un método EJB como un `observer` se lo puede anotar con **@Asynchronous**:

```
@Asynchronous
public void onCreditPaymentEvent(@Observes @Credit PaymentEvent event) {
```

Los métodos `observer` no tienen un orden de ejecución.

Para la implementación de un evento: una clase bean puede definir un evento, otro bean puede disparar el evento y otro bean puede controlar el evento. Los beans pueden estar en paquetes separados e incluso en capas separadas de la aplicación.

3.1.5 Interceptores en aplicaciones CDI

Los interceptores CDI constituyen un mecanismo estándar de Jakarta EE para implementar lógica transversal (cross-cutting concerns) de forma desacoplada y reutilizable. Un interceptor es una clase que se utiliza para interponerse en las invocaciones de métodos o eventos del ciclo de vida que ocurren en una clase de destino asociada. Permiten especificar el código para tareas en un solo lugar para facilitar el mantenimiento.³

Regla: Los interceptores CDI deben utilizarse para implementar lógica transversal reutilizable y desacoplada que afecte múltiples componentes de negocio, tales como: auditoría, logging, trazabilidad, validaciones técnicas, monitoreo. No se debe duplicar este tipo de lógica directamente en los servicios de negocio.

Utilidad: Permite centralizar funcionalidades transversales mediante mecanismos estándar de Jakarta EE, reduciendo duplicidad de código, mejorando mantenibilidad, reutilización y desacoplamiento de la lógica técnica respecto a la lógica funcional de negocio.

Consideraciones:

Al implementar interceptores se debe tomar en cuenta lo siguiente:

Los interceptores que son especificados en el `beans.xml` aplican solo a clases dentro del mismo módulo maven. Se debe usar la anotación **@Priority** para especificar interceptores a nivel global para una aplicación que contenga varios módulos, como en el siguiente ejemplo:

```
@Logged
```

³ https://jakarta.ee/learn/docs/jakartaee-tutorial/current/cdi/cdi-adv/cdi-adv.html#_using_interceptors_in_cdi_applications

```
@Interceptor
@Priority(Interceptor.Priority.APPLICATION)

public class LoggedInterceptor implements Serializable { ... }
```

Interceptores con menor prioridad son ejecutados primero. No se requiere especificar al interceptor en el beans.xml cuando se usa la anotación `@Priority`.

3.1.6 Decoradores en aplicaciones CDI

Los decoradores son externamente similares a los interceptores. Sin embargo, realizan tareas complementarias a las de los interceptores. Los interceptores realizan tareas transversales asociadas con la invocación de métodos y con el ciclo de vida de los beans, pero no pueden ejecutar lógica de negocio. Los decoradores, por otro lado, sí ejecutan lógica de negocio al interceptar los métodos de negocio de los beans. Su lógica es específica de una aplicación en particular, no transversal.⁴

Regla: Los decoradores CDI (`@Decorator`) deben utilizarse cuando sea necesario extender o complementar el comportamiento funcional de servicios de negocio mediante composición desacoplada, sin modificar la implementación original del componente. Su uso debe limitarse a escenarios donde se requiera agregar comportamiento adicional reutilizable sobre contratos definidos por interfaces.

Utilidad: Permite implementar extensiones funcionales de forma transparente y desacoplada, favoreciendo reutilización, mantenibilidad y cumplimiento del principio abierto/cerrado (Open/Closed Principle), evitando modificación directa de componentes de negocio existentes en aplicaciones Jakarta EE empresariales.

3.1.7 Estereotipos en aplicaciones CDI

En muchas aplicaciones, el uso de patrones arquitectónicos genera un conjunto de roles recurrentes para los beans. Un estereotipo permite al desarrollador identificar dicho rol y declarar metadatos comunes para los beans con ese rol en una ubicación centralizada.⁵

Regla: Cuando en la aplicación se tiene más de dos beans que tienen iguales características (combinaciones de mismo: scope, `@Named`, interceptor binding, u otro cualificador o anotación) se debe utilizar estereotipos creados por el desarrollador, para lo cual se debe utilizar la anotación `@Stereotype` (paquete `jakarta.enterprise.inject.Stereotype`).

CDI dispone de un estereotipo predefinido llamado `@Model` (paquete `jakarta.enterprise.inject.Model`) que está destinado a ser utilizado en beans que definen la capa del modelo en una arquitectura MVC. Tal estereotipo especifica que un bean tendría `@Named` y `@RequestScoped`. Entonces, si un bean requiere ser anotado con `@Name` y `@RequestScoped` se debe usar el estereotipo predefinido llamado `Model` y no crear uno nuevo.

```
@Model
public class MockLoginAction extends LoginAction { ... }
```

⁴ https://jakarta.ee/learn/docs/jakartaee-tutorial/current/cdi/cdi-adv/cdi-adv.html#_using_decorators_in_cdi_applications

⁵ https://jakarta.ee/specifications/cdi/4.0/jakarta-cdi-spec-4.0.html?utm_source=#stereotypes

Utilidad: Ayuda a la reutilización de anotaciones sin tener que escribirlas explícitamente al crear los beans. Los estereotipos pueden ser útiles en una aplicación en la que se tiene un número de beans que ejecutan iguales funciones.

Ejemplos:

- Se requiere crear un estereotipo de nombre Action, el mismo que tenga el scope `@RequestScoped`, `@Named` y el interceptor binding `@Logged`. Todos los beans anotados con `@Action` tendrían: request scope, el nombrado por default de EL, y el interceptor binding `@Logged`.

```
@Model
@Logged
@Stereotype
@Target(TYPE)
@Retention(RUNTIME)
public @interface Action {}
```

- Se requiere crear un estereotipo de nombre 'Simulacion', el mismo que tenga la anotación `@Alternative`. Todos los beans anotados con '`@Simulacion`' serían alternatives.

```
@Alternative
@Stereotype
@Target(TYPE)
@Retention(RUNTIME)
public @interface Simulacion {}
```

Notas:

Es posible aplicar varios estereotipos al mismo bean, para este caso al bean llamado 'MockLoginAction' se anotará con los estereotipos antes creados `@Action` y `@Simulacion`,

```
@Action
@Simulacion
public class MockLoginAction extends LoginAction { ... }
```

Si ya se tiene un estereotipo definido y solo se requiere modificar su scope o ámbito se debe **sobrescribir** el scope especificado en el estereotipo, simplemente especificando un diferente scope para el bean. Por ejemplo, al bean 'MockLoginAction' se le da el 'session scope' en vez de 'request scope' (el que está definido el estereotipo `@Action`).

```
@SessionScoped
@Action
@Simulacion
public class MockLoginAction extends LoginAction { ... }
```

3.2 Componentes de Negocio

3.2.1 Servicios CDI

Regla: Los componentes que implementen lógica de negocio estándar deben desarrollarse utilizando CDI como mecanismo principal para la gestión de componentes, ciclo de vida e inyección de dependencias, empleando gestión transaccional declarativas mediante `@Transactional` (*paquete: jakarta.transaction*). El uso de EJB `@Stateless` debe restringirse de acuerdo con la [regla](#).

Utilidad: Simplifica el modelo arquitectónico de la aplicación al unificar la gestión de componentes bajo CDI, reduciendo complejidad y acoplamiento innecesario a APIs EJB. Favorece mantenibilidad, pruebas e integración natural con JPA y JTA, alineándose con las prácticas recomendadas en Jakarta EE.

Regla: Los componentes de negocio CDI (`@Transactional`) deben diseñarse como componentes stateless (sin estado) y declarar explícitamente su *scope* utilizando las anotaciones del paquete `jakarta.enterprise.context`.

Se deberá utilizar preferentemente el *scope* `@ApplicationScoped`, para lo cual, el componente debe ser completamente stateless y thread-safe.

Nota: Un componente stateless No depende del estado interno de la instancia. Es aquel que no mantiene estado mutable asociado a usuarios, solicitudes o procesos dentro de sus atributos de instancia. Su comportamiento debe depender exclusivamente de parámetros de entrada, recursos inyectados o administrados por el contenedor y variables locales de ejecución.

Ejemplo correcto:

```
@ApplicationScoped
@Transactional
public class ClienteService {

    @PersistenceContext
    private EntityManager em;

    public Cliente buscar(Long id) {
        return em.find(Cliente.class, id);
    }
}
```

Ejemplo incorrecto: // Porque clienteActual sería compartido entre todos los usuarios.

```
@ApplicationScoped
@Transactional
public class ClienteService {

    private Cliente clienteActual;
}
```

Utilidad: Arquitectura desacoplada, escalable y segura para la ejecución concurrente, separando la lógica de negocio de la gestión del estado, mejorando la reutilización de los componentes y alineando la implementación con las prácticas recomendadas de Jakarta EE.

En la siguiente tabla se puede ver los diferentes scopes de `jakarta.enterprise.context`:

<i>Scope</i>	<i>Instancias</i>	<i>Casos de uso recomendados</i>
@Dependent	Una por cada inyección	Utilizar únicamente para componentes auxiliares o cuando el ciclo de vida deba depender del bean que los inyecta; no debería emplearse como scope por defecto en servicios de negocio.
@ApplicationScoped	Una única instancia para toda la aplicación	Scope predeterminado para servicios de negocio, fachadas, repositorios y endpoints REST/SOAP que sean completamente stateless.
@RequestScoped	Una instancia por petición HTTP	Cuando el componente necesite mantener información exclusivamente durante el ciclo de vida de una solicitud HTTP.
@SessionScoped	Una por sesión de usuario	Únicamente cuando sea imprescindible conservar estado específico del usuario entre múltiples solicitudes.
@ConversationScoped	Una por conversación (JSF)	Reservar para procesos conversacionales de varias vistas en JSF.

3.2.2 Session Beans

Regla: El uso del archivo ejb-jar.xml debe limitarse a escenarios de configuración externa, compatibilidad con aplicaciones legadas o requerimientos específicos de despliegue y administración EJB.

Utilidad: Simplificar la configuración de aplicaciones, reducir complejidad de despliegue y favorecer el modelo moderno basado en anotaciones, manteniendo compatibilidad con escenarios legados donde se requiera configuración declarativa externa.

Regla: Stateless Session Beans. – El uso de componentes EJB @Stateless debe restringirse a escenarios cuando la aplicación requiera capacidades avanzadas propias del modelo Enterprise Beans, incluyendo procesamiento asíncrono (@Asynchronous), timers EJB, EJBs remotos o integración con componentes legados. No debe ser utilizado @Stateless en los servicios web.

Regla: Stateful Session Beans. - Los EJB @Stateful deben utilizarse en escenarios que requieran mantener estado conversacional asociado a un cliente entre múltiples invocaciones. Su uso debe ser analizado cuidadosamente por el desarrollador y en el caso de requerir soporte comunicarse con el Arquitecto TI asignado.

Regla: Singleton Session Beans. - Los Singleton Session Beans (@Singleton) deben utilizarse en escenarios donde sea necesario mantener un estado compartido y controlado a nivel de aplicación, administrar recursos únicos concurrentes o ejecutar procesos asociados al ciclo de vida de la aplicación.

Nota: Considerar el uso de Singleton Session Bean en ambientes de alta disponibilidad (HA), con el objetivo de que los valores entre los servidores estén sincronizados mediante mecanismos de caché, y en el caso de requerir soporte, consultar con el Arquitecto TI asignado.

Utilidad: Centralizar recursos y estados compartidos administrados por el contenedor Jakarta EE, garantizando control de concurrencia, acceso coordinado entre múltiples hilos y administración segura del ciclo de vida de componentes empresariales críticos.

3.2.3 Message-Driven Bean (MDB)

Regla: Los Message-Driven Beans (MDB) deben utilizarse para el procesamiento asíncrono basado en mensajería administrada por el contenedor Jakarta EE, especialmente en escenarios de integración empresarial, consumo de colas o tópicos JMS, desacoplamiento de procesos, procesamiento distribuido y manejo de operaciones de larga duración o alta concurrencia orientadas a eventos.

Utilidad: Permite implementar procesamiento asíncrono confiable y desacoplado mediante mensajería empresarial, favoreciendo escalabilidad, tolerancia a fallos, integración entre sistemas y administración transaccional automática de mensajes bajo el contenedor Jakarta EE.

Regla: En entornos clusterizados, los Message-Driven Beans (MDB) deben desplegarse y configurarse en los nodos participantes del cluster para garantizar procesamiento distribuido, balanceo de carga y alta disponibilidad de mensajes JMS.

Utilidad: Permite asegurar continuidad operativa, procesamiento concurrente y distribución eficiente de carga en arquitecturas empresariales basadas en mensajería asíncrona Jakarta EE.

Importante: Las reglas aplicables a la mensajería Jakarta Messaging se encuentran en la sección **Mensajería**

3.2.4 Transacciones

Regla: Los componentes que implementen lógica de negocio estándar, la gestión transaccional debe realizarse mediante transacciones declarativas usando la anotación `@Transactional` (paquete: `jakarta.transaction`).

Utilidad: Permite administrar de manera uniforme y desacoplada los límites transaccionales de la lógica de negocio, reduciendo complejidad técnica y favoreciendo consistencia, mantenibilidad e integración con JPA y otros recursos transaccionales administrados por Jakarta EE.

En aplicaciones Jakarta EE, una transacción es una serie de tareas que deben ser realizadas juntas y de forma atómica.

- Si una o más tareas **falla**: *'roll-back'*
- Si todas las tareas son **exitosas**: *'commit'*

Reglas:

- Para los enterprise beans de session o message driven, se debe usar transacciones gestionadas por el contenedor (CMT – Container Managed Transaction). Solo en casos

donde se haya hecho un previo análisis de su uso se puede implementar en los session beans las transacciones gestionadas por el bean (BMT - Bean Managed Transaction).

- En servicios de negocio EJB que requieran administración transaccional mediante Container-Managed Transactions (CMT), el alcance y propagación de las transacciones deben definirse explícitamente utilizando la anotación `@TransactionAttribute` (`jakarta.ejb.TransactionAttribute`) y los atributos transaccionales apropiados (`REQUIRED`, `REQUIRES_NEW`, `MANDATORY`, `NOT_SUPPORTED`, `SUPPORTS`, `NEVER`). La selección del atributo transaccional debe realizarse de acuerdo con el comportamiento funcional y el contexto transaccional requerido por cada operación de negocio.
- En componentes de negocio CDI que requieran administración transaccional declarativa, el alcance y propagación de las transacciones deben definirse explícitamente mediante la anotación `@Transactional` y los tipos transaccionales (`Transactional.TxType.REQUIRED`, `REQUIRES_NEW`, `MANDATORY`, `NOT_SUPPORTED`, `SUPPORTS`, `NEVER`). La selección del tipo transaccional debe realizarse de acuerdo con el comportamiento funcional y el contexto transaccional requerido por cada operación de negocio.

A continuación, se describen la descripción de los atributos o tipos transaccionales:

REQUIRED: Es el tipo transaccional recomendado y predeterminado para métodos de negocio tanto en EJB como en servicios CDI con `@Transactional`. El método debe ejecutarse dentro de una transacción existente o iniciar una nueva si no existe una transacción activa. Debe utilizarse como configuración estándar para operaciones de negocio transaccionales, salvo que exista un requerimiento funcional o técnico que justifique explícitamente otro comportamiento transaccional.

REQUIRES_NEW: Debe utilizarse cuando una operación necesite ejecutarse obligatoriamente en una nueva transacción independiente de la transacción invocadora. La transacción actual, si existe, será suspendida temporalmente mientras se ejecuta la nueva transacción. Es apropiado para procesos que deban persistirse independientemente del resultado de la transacción principal, tales como auditoría, trazabilidad, bitácoras, notificaciones críticas o registro de errores.

MANDATORY: Debe utilizarse únicamente cuando el método requiera ejecutarse obligatoriamente dentro de una transacción previamente iniciada por el componente invocador. Si el método es invocado sin un contexto transaccional activo, el contenedor generará una excepción. Se recomienda para operaciones que forman parte obligatoria de un flujo transaccional de negocio ya existente.

NOT_SUPPORTED: Debe utilizarse en métodos que no requieran participación transaccional. Si existe una transacción activa, ésta será suspendida temporalmente durante la ejecución del método. Es apropiado para operaciones de lectura no críticas, consultas de catálogos estáticos, envío de correos, generación de reportes o procesos técnicos que no deban formar parte de la transacción de negocio. Su uso puede reducir overhead transaccional y mejorar rendimiento.

SUPPORTS: Indica que el método puede ejecutarse dentro o fuera de una transacción dependiendo del contexto del invocador. Si existe una transacción activa, el método

participará en ella; de lo contrario, se ejecutará sin transacción. Debido a que introduce comportamiento transaccional variable y potencialmente ambiguo, su uso debe restringirse únicamente a escenarios técnicamente justificados, principalmente en operaciones de solo lectura o servicios auxiliares.

NEVER: Debe utilizarse únicamente en métodos que no deban ejecutarse bajo ningún contexto transaccional. Si el método es invocado dentro de una transacción activa, el contenedor generará una excepción. Se recomienda para operaciones técnicas o integraciones externas incompatibles con contextos transaccionales.

3.2.5 Rollback de transacciones gestionadas por contenedor

Cuando el contenedor EJB lanza excepciones del tipo *java.lang.RuntimeException*, *jakarta.ejb.EJBException* (excepciones no-chequeadas) que son excepciones del sistema tales como errores de conexión de bases de datos o de red, el contenedor fuerza un rollback automático sobre toda la transacción; pero si el enterprise bean lanza una excepción de aplicación (excepciones chequeadas) el rollback no es automático.

Regla: En un método de un enterprise bean que lance una excepción de aplicación (*excepción que ocurre mientras se ejecuta lógica de negocio*) y para marcar a la transacción para que haga rollback, en la captura de la excepción se debe usar el método *setRollbackOnly()* de la interface *jakarta.ejb.EJBContext*.

Ejemplo:

```
@Stateless
@LocalBean
public class SujetoControlService {
    @Resource
    private EJBContext sessionCtx;
    public void validarFuenteSujetoControlEsEliminable(FuenteSujetoControl fuenteSujetoControlAValidar,
    Preoperativo operativo) throws ControladaException {
        List<FuenteSujetoControl> listaFuentesSujetoControlIntervendas =
            intervencionDao.obtenerFuentesIntervenidasPorPreoperativo(operativo);
        boolean respuestaValidacion = true;
        for(FuenteSujetoControl fuenteSujetoConIntervencion: listaFuentesSujetoControlIntervendas)
        {
            if(fuenteSujetoControlAValidar.getCodigo().equals(fuenteSujetoConIntervencion.getC
            odigo())) {
                respuestaValidacion = false;
                break;
            }
        }
        if (!respuestaValidacion) {
            String mensaJava EExcepcion = MensajeUsuarioUtil.obtenerMensaJava
            EExcepcion("menaJava EExcepcion");
            if (!sessionCtx.getRollbackOnly()) {
                sessionCtx.setRollbackOnly();
                throw new ValidacionNegocioException(mensaJava EExcepcion);
            }
        }
    }
}
```

```
}
```

Consideración. - Hay varios buenos motivos para que las transacciones funcionen así. Las excepciones chequeadas pueden usarse para identificación de eventos o para redirigir el procesamiento basándose en ciertas condiciones. Pero, además, el código fuente de la aplicación podría tomar medidas correctivas en algunas excepciones chequeadas, y permitir que la transacción se complete. Por ejemplo, en una aplicación bancaria, para completar la transacción de transferencia de fondos se necesita enviar una auditoría como parte del proceso, al inicio y al final de la transacción. Si la transacción de la transferencia de fondos fallara (errores de aplicación o de infraestructura) se lanzará algún tipo de excepción chequeada indicando que la transacción no pudo ejecutarse.

Si las excepciones chequeadas causaran un rollback automático, toda la transferencia de fondo tendría un rollback. Al no realizar un rollback automático sobre excepciones chequeadas, es posible capturar la excepción y realizar alguna acción correctiva (como enviar la auditoría de la transacción).

Regla: Cuando no se quiera reenviar nuevamente el mensaje a la cola por un *RuntimeException* o por un error de la lógica de la aplicación, se debe utilizar un método helper con un atributo de tracción `REQUIERES_NEW` para que se ejecute en un contexto de transacción diferente y en función de ese contexto manejar las excepciones causadas por la lógica del procesamiento.

Utilidad: Debido a que los MDB son por defecto transaccionales y hay un error de lógica en el proceso hay que separar la transacción actual de la transacción del procesamiento para evitar el re-encolamiento infinito de los mensajes.

Consideraciones:

Es una buena práctica establecer tiempos de expiración de los mensajes o mantener un control sobre el número de veces que un mensaje puede ser re encolado.

Regla: El rollback automático de la transacción de los componentes de negocio CDI (`@Transactional`) debe ser administrado declarativamente por JTA mediante excepciones de negocio que heredan de *RuntimeException*.

3.3 Java Persistencia API - JPA

Regla: Para el acceso a datos se debe utilizar Java Persistencia API, esto para: mapear estructuras de base de datos, ejecutar cualquier sentencia DML (Data Manipulation Language) o ejecución de algún procedimiento almacenado o paquete. Solo se debe utilizar JDBC en sentencias DML, procedimientos o paquetes que tengan que ver con gran cantidad de datos y las mismas estén dando problemas de rendimiento; el Arquitecto TI asignado debe validar la factibilidad de uso de este mecanismo y garantizar la reutilización de componentes genéricos.

Utilidad: Java Persistencia API se utiliza para mapear bases de datos relacionales a través de POJOs. JPA no se limita a los componentes empresariales EJB, también se puede utilizar en aplicaciones web y de escritorio.

Regla: Los mecanismos de acceso al repositorio de datos denominados JPQL y Criteria API se puede usar considerando lo siguiente:

- ✓ Para el uso de Criteria API:
 - Consultas dinámicas donde los parámetros y estructura de la consulta varían dependiendo de las condiciones en tiempo de ejecución se debe implementar con Criteria API.
 - Consultas en la cuales se realicen joins con más de 2 tablas.
 - Consultas que impliquen el uso de subqueries.
- ✓ Para la construcción de consultas estáticas es decir que no cambian dependiendo de los parámetros se debe usar JPQL.

Para revisar las reglas de estos mecanismos verifique las secciones [Java Persistence Query Language \(JPQL\)](#) y [Criteria API](#).

Utilidad: El API de Criteria Query ofrece ventajas de mantenibilidad y orden basado en POO para construcción de consultas dinámicas, mientras que el JPQL dado a su simplicidad es más usable en consultas estáticas.

Regla: Para las consultas JPA que devuelven **obligatoriamente** cero o un único registro se debe utilizar el método **getSingleResultOrNull()**, generalmente consultas por clave primaria; la posible existencia de más de un resultado se considera como una condición excepcional de unicidad no cumplida que debe ser detectada y gestionada.

Ejemplo:

```
Usuario usuario = em.createQuery("""
    SELECT u
    FROM Usuario u
    WHERE u.codigo = :codigo
    """, Usuario.class)
    .setParameter("codigo", codigo)
    .getSingleResultOrNull();

return Optional.ofNullable(usuario);
}
```

Utilidad: Permitir que la consulta exprese explícitamente que se espera como máximo un único resultado, diferenciando de forma clara la ausencia de registros de una violación de la condición de unicidad y evita el uso innecesario de `getResultList()` para consultas de cardinalidad único.

Regla: Para las consultas JPA que devuelven uno o varios registros se debe utilizar el método **getResultList()**. Si se necesitan realizar validaciones dependiendo del número de registros retornados se deben realizar en la capa de servicios exceptuando los casos en que dichas validaciones produzcan un cambio de flujo de ejecución para realizar operaciones en la misma capa (Repository/DAO).

Utilidad: Garantizar el manejo adecuado de consultas que pueden devolver múltiples registros, manteniendo en el Repository/DAO la responsabilidad de acceso y recuperación de datos, y

trasladando a la capa de servicios las validaciones relacionadas con la cantidad de registros cuando estas correspondan a reglas de negocio.

Ejemplo

```
@ApplicationScoped
public class ComprobanteRepository {
    public List<Comprobantes> buscarPorTipo(Long tipoId) {

        return em.createQuery("""
        SELECT c
        FROM Comprobantes c
        WHERE c.tipo.id = :tipoId
        """, Comprobantes.class)
        .setParameter("tipoId", tipoId)
        .getResultList();
    }
}
```

Regla: Las consultas deben considerar la paginación acorde a lo descrito en el Estándar Desarrollo Aplicaciones Web. (Ver ruta en [Anexo 3: Recursos relacionados](#)).

Regla: La anotación @Version solo puede utilizarse sobre un atributo de tipo Long declarado en la entidad la misma que representa un campo de tipo number en el modelo de negocio.

Pool de conexiones

Aplicar lo indicado en el documento '*Estándar de Configuración de Servidor de Aplicaciones*'. (Ver ruta en [Anexo 3: Recursos relacionados](#))

3.3.2 Unidad de persistencia

La unidad de persistencia define todas las clases de entidad que son gestionadas por la instancia del EntityManager en una aplicación. Este conjunto de clases de entidad representa las estructuras contenidas en una única base de datos o esquema.

Regla: Las unidades de persistencia se deben definir en el archivo de configuración ***persistence.xml***, el cual debe estar ubicado dentro del proyecto a nivel de los fuentes de las clases de entidad dentro de la carpeta ***/src/main/resources/META-INF***.

En el documento '*Estándar de Desarrollo de Aplicaciones – Estructura y Nomenclatura*' (Ver ruta en [Anexo 3: Recursos relacionados](#)) se puede revisar el detalle de la nomenclatura de la unidad de persistencia.

Ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_1.xsd"
version="3.1">
<persistence-unit name="ProyectoBasePU"
    transaction-type="JTA">
    <jta-data-source>java:/ProyectoBaseDS</jta-data-source>
    <jar-file>sri-base-modelo-2.0.0.jar</jar-file>
    <class>ec.gob.sri.base.modelo.TipoParametro</class>
    <properties>
        <property name="hibernate.show_sql" value="false" />
        <property name="hibernate.format_sql" value="false" />
    </properties>
</persistence-unit>
</persistence>
```

Dónde:

jta-data-source: es el nombre global del JNDI de la fuente de datos (datasource) configurado en la sección Pool de conexiones.

jar-file: es el archivo .jar que contiene las entidades de persistencia de la unidad de persistencia configurada.

hibernate.show_sql - hibernate.format_sql: para estas propiedades, utilizar con valor 'true' solo para pruebas, para producción deben tener valor 'false'.

Nota: Otras propiedades que pueden incluirse son aquellas para el caché de segundo nivel, revisar la sección [3.3.9 Caché de segundo nivel](#).

Regla: El archivo persistence.xml sólo debe contener la unidad de persistencia del dominio de negocio al cual pertenece. De ser necesaria la inclusión de unidades de persistencia adicionales revisarlo con el Arquitecto TI asignado.

3.3.3 Java Persistence Query Language (JPQL)

JPQL permite escribir sentencias DML (Data Manipulation Language) portables que funcionan independientemente del almacén de datos subyacente.

Reglas:

- Se deben utilizar consultas nativas (**NativeQuery**) solo cuando existan transacciones con movimiento de datos masivos en operaciones de inserción, actualización o borrado y se tengan problemas de rendimiento; el Arquitecto TI asignado debe validar la factibilidad de uso de este mecanismo y garantizar la reutilización de componentes genéricos.

- Si se requiere utilizar en las consultas los operadores como LIKE, OR, operadores de agregación, y junturas de más de cuatro tablas. que reducen el rendimiento, es necesario validar previamente con el Arquitecto TI asignado para determinar su viabilidad o alternativas de uso.
- Las consultas JPQL deben estar definidas como NamedQueries.

Ejemplo:

Se tiene la clase de entidad llamada '*VendorPart*' donde se requiere hacer una consulta nombrada (named query) que haga la sumatoria del valor de los precios de las partes que ha vendido un determinado id '*vendorId*' de vendedor:

```
@Entity
@Table(name="PERSISTENCE_ORDER_VENDOR_PART")
@NamedQuery(
    name="«findTotalVendorPartPricePerVendor",
    query="SELECT SUM(vp.price) " +
    "FROM VendorPart vp " +
    "WHERE vp.vendor.vendorId = :id"
)
public class VendorPart implements java.io.Serializable {
    private Long vendorPartNumber;
    private String description;
    private double price;
    private Part part;
    private Vendor vendor;
    public VendorPart() {}
}
```

3.3.4 Critería API

El Critería API permite definir sentencias DML (Data Manipulation Language) para las entidades mediante la creación de objetos que define la consulta. Los criterios de consulta se escriben utilizando las API de Java, son typesafe y portables. Tales consultas funcionan independientemente del almacén de datos subyacente.

Regla: Las consultas dinámicas implementadas mediante JPA Critería API deben utilizar metamodelos estáticos generados por la especificación JPA (Static Metamodel) para la referencia tipada de atributos y entidades. No se debe acceder a atributos de entidades mediante nombres literales (String) dentro de expresiones Critería.

Utilidad: Construir consultas tipadas, seguras y mantenibles, reduciendo errores en tiempo de ejecución asociados a nombres de atributos inválidos y facilitando refactorización, validación en compilación y compatibilidad con herramientas de desarrollo Jakarta EE.

Ejemplo:

```
CriteriaBuilder cb = session.getCriteriaBuilder();
```

```
CriteriaQuery<Book> criteriaQuery = cb.createQuery(Book.class);
Root<Book> root = criteriaQuery.from(Book.class);
criteriaQuery.select(root).where(cb.equal(root.get(Book_.gradYear), 2025));

Query<Book> query = session.createQuery(criteriaQuery);
List<Book> results = query.getResultList();
```

Regla: Los metamodelos estáticos se deben crear automáticamente con el artefacto de Maven '[hibernate-jpamodelgen-jakarta](#)' ya configurado en el proyecto Jakarta EE. La creación manual de los metamodelos se lo puede hacer siempre y cuando la creación automática no esté disponible en la aplicación.

Ejemplo: Este es un ejemplo de consulta para ser ejecutada con Criteria API debido a que tiene opciones de varios criterios usando el API de Java en lugar de hacer con cadenas que es propio de JPQL:

```
public List<Employee> buscarEmpleados(String name, String deptName,
    String projectName, String city) {
    CriteriaBuilder cb = em.getCriteriaBuilder();
    CriteriaQuery<Employee> c = cb.createQuery(Employee.class);
    Root<Employee> emp = c.from(Employee.class);
    c.select(emp);
    c.distinct(true);

    Join<Employee, Project> project = emp.join("projects", JoinType.LEFT);
    List<Predicate> criteria = new ArrayList<Predicate>();

    Map<String,String> mapaCriteriosValores = crearCriteriosValores(name, deptName,
        projectName, city);
    Iterator<String> clavesCriterios= mapaCriteriosValores.keySet().iterator();

    while(clavesCriterios.hasNext()){
        String claveCriterio = clavesCriterios.next();
        if (claveCriterio.equals("dept")) {
            ParameterExpression<String> p = cb.parameter(String.class, claveCriterio);
            criteria.add(cb.equal(emp.get(claveCriterio).get("name"), p));
        }
        if (claveCriterio.equals("project")) {
            ParameterExpression<String> p = cb.parameter(String.class, claveCriterio);
            criteria.add(cb.equal(project.get("name"), p));
        }
        if (claveCriterio.equals("city")) {
            ParameterExpression<String> p = cb.parameter(String.class, claveCriterio);
            criteria.add(cb.equal(emp.get("address").get(claveCriterio), p));
        }
        if (claveCriterio.equals("name")) {
            ParameterExpression<String> p = cb.parameter(String.class, claveCriterio);
            criteria.add(cb.equal(emp.get(claveCriterio), p));
        }
    }
}
```

Ejemplo de uso de JPA Criteria API

Regla: Si se requieren utilizar en las consultas los operadores como LIKE, OR, operadores de agregación, y juntas de más de cuatro tablas que reducen el rendimiento, se debe validar previamente con el Arquitecto TI asignado para su viabilidad o alternativas de uso.

3.3.5 Ejecución de procedimientos almacenados

Regla: La invocación de procedimientos almacenados desde JPA, requiere la revisión y aprobación del Arquitecto TI asignado.

Regla: Para los escenarios aprobados de ejecución de procedimientos almacenados estos no deben contener lógica compleja y su tiempo de procesamiento debe ser menor o igual acorde a

lo configurado en el atributo default-timeout del subsistema de transacciones del servidor de aplicaciones.

Regla: Para los escenarios aprobados de ejecución de un procedimiento almacenado tanto si retorna o no parámetros, se debe utilizar JPQL, para lo cual usar el objeto *StoredProcedureQuery*. Excepto para lo descrito en la sección JPA respecto a JDBC.

Ejemplo:

Se requiere implementar un procedimiento almacenado de nombre 'SP_BUSCAR_VENDOR_POR_ID' que reciba un parámetro de entrada (tipo entero) y otro de salida (tipo cadena), la respuesta del procedimiento debe ser el nombre del vendedor (tabla PERSISTENCE_ORDER_VENDOR) que corresponde al id que es dato de entrada. El procedimiento debe ser ejecutado mediante JPQL, en un método que responda lo que devuelva el procedimiento.

Método	Procedimiento Almacenado
<pre> public String buscarVendorByNombre(String id) { <i>StoredProcedureQuery</i> storedProcQuery = em.createStoredProcedureQuery("SP_BUSCAR_ VENDOR_POR_ID"); storedProcQuery.registerStoredProcedurePara meter(1, Integer.class, ParameterMode.IN); storedProcQuery.registerStoredProcedurePara meter(2, String.class, ParameterMode.OUT); storedProcQuery.setParameter(1,Integer.parseInt(id)); boolean result = storedProcQuery.execute(); String nombreResp = (String) storedProcQuery.getOutputParameterValue(2) ; return nombreResp; } </pre>	<pre> CREATE OR REPLACE PROCEDURE SP_BUSCAR_VENDOR_POR_ID(psi_id IN number, pso_nombre OUT VARCHAR2) AS var_mensaje_error VARCHAR2 (500); BEGIN SELECT v.VENDORNAME INTO pso_nombre FROM PERSISTENCE_ORDER_VENDOR v WHERE v.VENDORID=psi_id; EXCEPTION WHEN OTHERS THEN raise_application_error(-20001,'Error SP_BUSCAR_VENDOR_POR_ID -' SQLCODE ':Causado por ==>' var_mensaje_error); END; </pre>

3.3.6 Entidades

Regla: Para el desarrollo de las clases de entidad, se deben seguir los siguientes lineamientos:

1. Las anotaciones JPA deben ser puestas sobre los atributos los mismos que hacen referencia a los campos de la tabla mapeada.
2. Todos los tipos de datos de los atributos deben ser tipo objeto, es decir, no utilizar los tipos primitivos java (int, double, boolean, etc.). Así mismo los atributos que corresponden a números decimales deben ser del tipo BigDecimal, y los atributos que corresponden a números enteros serán del tipo Long.
3. Cada entidad debe tener id o clave primaria (primary key) y para denotar a un atributo o campo como clave primaria se debe usar la anotación *@Id* (paquete: jakarta.persistence.Id). El nombre del atributo de la clave primaria debe ser '*codigo*' y tipo de dato '*Long*'.
4. No se permite el uso de claves primarias compuestas (primary key), a excepción de mapeos a aplicaciones legadas donde se tengan claves primarias compuestas.

- Para la generación de los ids o claves primarias se debe usar la estrategia de secuencias '**SEQUENCE**' (**strategy** = GenerationType.SEQUENCE) mediante la creación de una secuencia de base de datos. En la anotación @SequenceGenerator se debe poner el atributo '**allocationSize**' con valor '1'.

Ejemplo:

```
@Entity
@Table(name = "FFC_ARCHIVO_SUJETO_REQUIERE_AP")
public class ArchivoSujetoRequiereAprobacion implements Serializable {
    public ArchivoSujetoRequiereAprobacion(){}
    @Id
    @SequenceGenerator(name = "FFC_SEQ_ARCHIVO_SUJETO_REQUIER", sequenceName =
        "FFC_SEQ_ARCHIVO_SUJETO_REQUIER", allocationSize=1 )
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
        "FFC_SEQ_ARCHIVO_SUJETO_REQUIER")
    @Column(name = "CODIGO_ARCHIVO_SUJETO_REQUIERE_AP")
    private Long codigo;
}
```

3.3.7 Clases embeddable en entidades

Regla: Se deben utilizar clases @Embeddable en entidades JPA cuando sea necesario modelar atributos compuestos reutilizables, representar conceptos de dominio con granularidad fina o encapsular grupos coherentes de atributos relacionados dentro de una entidad. Las clases embeddable deben utilizarse especialmente en los siguientes escenarios:

- Más de una entidad compartan una misma estructura lógica de atributos.
- Por ampliación del dominio de la entidad se requiere agregar nuevos atributos donde por granularidad fina en el modelado del diseño se puede utilizar una clase 'embeddable' para estos nuevos atributos.
- El diseño relacional no se ajusta al modelado orientado a objetos (por ejemplo, una vista que involucra varias tablas se puede representar en varios objetos).

Regla: No se deben utilizar clases 'embeddables' para solucionar problemas de PMD (regla TooManyFields) en entidades.

Regla: Las entidades JPA y clases @Embeddable deben utilizar @Access(AccessType.FIELD) como estrategia estándar de acceso persistente. La incorporación de clases embeddable dentro de entidades JPA debe realizarse mediante la anotación @Embedded (jakarta.persistence.Embedded). Tomar en cuenta que con AccessType.FIELD el mapeo de las columnas (@Column) es sobre los atributos de persistencia,

Utilidad: Estandarizar el acceso a atributos persistentes, garantizar consistencia en el mapeo JPA y mejorar mantenibilidad y claridad del modelo de entidades en aplicaciones Jakarta EE.

Ejemplo:

Se tienen las Entidades JPA de nombres 'Order' y 'User', las cuales tienen dos campos iguales o similares referentes a la dirección () y no se quiere duplicar los mismos campos en las Entidades. Para solucionar esto se debe crear una clase Embeddable (insertable) de nombre 'Address' y luego sea insertada en las entidades mencionadas. En el ejemplo, para crear la clase Embeddable, se debe tener en cuenta que para la entidad 'Order' los nombres de las columnas

referentes a la dirección son: ***address_street*** y ***codePostal*** y para la entidad 'User' los nombres de las columnas referentes a la dirección son: ***user_street*** y ***user_postal***:

Entidad Order	Entidad User
<pre> @Entity @Table(name = "ORDER") public class Order implements Serializable { @Id @Column(name = "CODIGO_ORDER") private Long id; public Long getId() { return id; } public void setId(Long id) { this.id = id; } @Embedded private Address address; public Address getAddress() { return address; } public void setAddress() { this.address = address; } } </pre>	<pre> @Entity public class User implements Serializable{ @Id @Column(name = "CODIGO_USER") private Long id; public Long getId() { return id; } public void setId(Long id) { this.id = id; } @Embedded @AttributeOverrides({ @AttributeOverride(name="street", column = @Column(name="user_street")), @AttributeOverride(name="codePostal", column = @Column(name="user_postal")) }) private Address address; public Address getAddress() { return address; } public void setAddress() { this.address = address; } } </pre>
Entidad Address (Embedded)	
<pre> @Embeddable @Access(AcessType.FIELD) private class Address{ @Column (name="address_street") private String street; @Column (name="code_postal") private int codePostal; public String getStreet() { return street; } public void setStreet() { this. street = street; } public String getCodePostal() { return codePostal; } public void setCodePostal() { this. codePostal = codePostal; } } </pre>	

3.3.8 Mapeo de relaciones entre entidades

Para que se pueda hacer el mapeo de relaciones entre entidades correctamente, la multiplicidad (one-to-one, many-to-one, one-to-many, many-to-many) y tipo de dirección (bidireccional y unidireccional) entre entidades de la unidad de persistencia, el modelado de base de datos debe estar bien definido.

Relaciones bidireccionales

Una relación bidireccional tiene tanto un lado propietario (padre) y un lado inverso (hijo), las mismas en su mapeo deben seguir las siguientes reglas:

1. El lado del hijo debe referirse a su lado padre usando el elemento '**mappedBy**' de las anotaciones `@OneToOne`, `@OneToMany`, o `@ManyToMany`. El elemento `mappedBy` designa la propiedad o campo en la entidad que es titular o padre de la relación.
2. En el lado de '**many**' en la relación bidireccional '**many-to-one**' no se debe definir el elemento '`mappedBy`'. El lado '**many**' es siempre el lado padre de la relación.
3. Para la relación bidireccional '**one-to-one**', el lado padre debe corresponder al lado que contiene el correspondiente '**foreign key**'.
4. En la entidad padre se debe definir el nombre de la columna que contendrá la clave foránea mediante la anotación '`@JoinColumn`'.
5. Para la relación bidireccional '**many-to-many**', cualquiera de los dos lados de la relación puede ser el lado propietario o padre. Antes de usar este tipo de relación en el mapeo de entidades, se debe validar con el Arquitecto TI asignado la factibilidad de uso.

Relaciones unidireccionales

En una relación unidireccional, solo una entidad debe tener un campo o propiedad de relación que se refiere a la otra entidad.

Lectura o recuperación de data de entidades

Para la recuperación de datos de las entidades de la unidad de persistencia, se deben seguir las siguientes reglas:

1. Para las relaciones entre entidades '**one-to-one**' (`@OneToOne`) y '**many-to-one**' (`@ManyToOne`) se debe utilizar el tipo de lectura **LAZY**.

Ejemplo:

```
@ManyToOne(fetch=FetchType.LAZY)
public VendorPart getVendorPart() {
    return vendorPart
}
```

Para las relaciones entre entidades '**one-to-many**' (`@OneToMany`) y '**many-to-many**' (`@ManyToMany`) se debe utilizar el tipo de lectura **LAZY** (valor por defecto).

Se debe usar la cláusula '**JOIN FETCH**' de JPQL cuando se requiera cargar de forma 'EAGER' alguna colección de algún tipo de relación con el tipo de lectura 'LAZY'; hacer esto siempre y cuando el tipo de relación '`@OneToOne`' o '`@OneToMany`' no tiene el atributo '`cascade`', caso contrario los datos de la colección si están disponibles.

En el ejemplo siguiente se requiere consultar todos los datos de '**LogDenuncia**' y cargar de forma 'EAGER' los datos de la colección de '**DetalleLogDenuncia**':

Entidad Propietario (padre)	Entidad Inverso (hijo)
<pre>@Entity @Table(name = "FFC_LOG_DENUNCIA") public class LogDenuncia implements Serializable { @Id</pre>	<pre>@Entity @Table(name = "FFC_DETALLE_LOG_DENUNCIA") public class DetalleLogDenuncia implements Serializable { ...</pre>

<pre> @Column(name = "CODIGO_LOG_DENUNCIA") private Long codigo; @OneToMany(mappedBy = "logDenuncia") private List<DetalleLogDenuncia> listaDetalleLogDenuncia; } SELECT log FROM LogDenuncia log JOIN FETCH log.listaDetalleLogDenuncia; </pre>	<pre> @JoinColumn(name = "CODIGO_LOG_DENUNCIA", referencedColumnName="CODIGO_LOG_DENUNCIA") @ Many ToOne(fetchType.LAZY) private LogDenuncia logDenuncia; } </pre>
---	--

Para las relaciones entre entidades 'one-to-many' (@OneToMany) y 'many-to-many' (@ManyToMany), se puede establecer a EAGER en alguna relación que se prevea que no pueda afectar al rendimiento de la aplicación y base de datos.

Entidades y operaciones en cascada

En el mapeo de las relaciones entre entidades la opción 'cascade' permite realizar operaciones en cascada sobre las relaciones, para lo cual se debe tomar en cuenta lo siguiente:

1. Utilizar el atributo '**cascade**' solo en relaciones dependientes (padre-hijo) por ejemplo la relación entre Libro y Copia es dependiente, si no existe el libro no tiene sentido que haya copias de este. Aplicando la opción *cascade* correcta, permitiría que operaciones sobre el padre se propaguen hacia el hijo.

Se debe utilizar el atributo 'cascade' solo en los tipos de relación 'one-to-one' (@OneToOne) y 'one-to-many' (@OneToMany).

Solo se debe utilizar el tipo de cascada (*CascadeType*) 'PERSIST'.

Ejemplo:

Se requieren mapear las relaciones bidireccionales de las entidades llamadas 'LogDenuncia' (hijo) y 'DetalleLogDenuncia' (padre), donde la relación entre las entidades es 'one-to-many' y además se requiere que se utilice el atributo 'cascade' '**PERSIST**'

Entidad Propietario (padre)	Entidad Inverso (hijo)
<pre> @Entity @Table(name = "FFC_LOG_DENUNCIA") public class LogDenuncia implements Serializable { @Id @SequenceGenerator(name = "FFC_SEQ_LOG_DENUNCIA", sequenceName = "FFC_SEQ_LOG_DENUNCIA", allocationSize = 1) @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "FFC_SEQ_LOG_DENUNCIA") @Column(name = "CODIGO_LOG_DENUNCIA") private Long codigo; @OneToMany(cascade = CascadeType.PERSIST, mappedBy = "logDenuncia") private List<DetalleLogDenuncia> listaDetalleLogDenuncia; </pre>	<pre> @Entity @Table(name = "FFC_DETALLE_LOG_DENUNCIA") public class DetalleLogDenuncia implements Serializable { @JoinColumn(name = "CODIGO_LOG_DENUNCIA", referencedColumnName = "CODIGO_LOG_DENUNCIA") @ManyToOne(fetch=FetchType.LAZY) private LogDenuncia logDenuncia; } </pre>

}	
---	--

3.3.9 Caché de segundo nivel

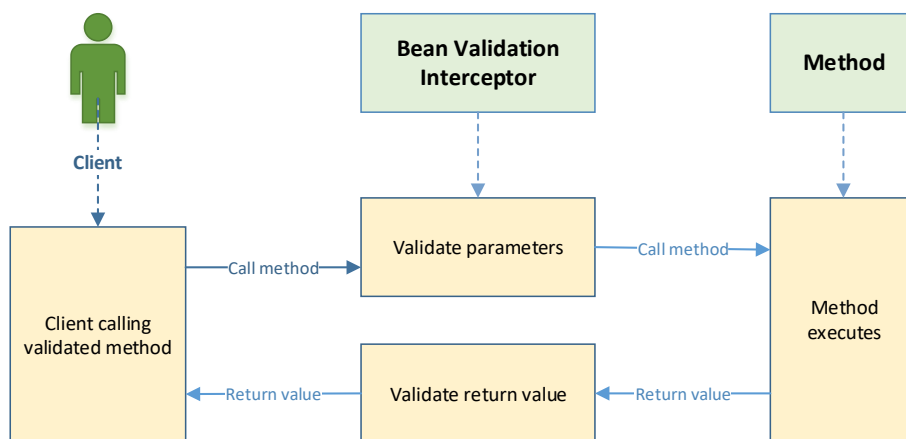
Regla: Para la caché de los datos de los catálogos, y datos de los parámetros, referirse a la [regla](#) correspondiente.

Regla: Para la implementación de caché de segundo nivel se debe usar la implementación de Infinispan del servidor de aplicaciones, ejemplo.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_1.xsd"
    version="3.1">
<persistence-unit name="CatastroPU" transaction-type="JTA">
<jta-data-source>java:/CatastroDS</jta-data-source>
<jar-file>sri-catalogo-modelo-1.0.0-SNAPHOST.jar</jar-file>
<properties>
<property name="hibernate.show_sql" value="false" />
<property name="hibernate.format_sql" value="false" />
<property name="hibernate.cache.use_second_level_cache" value="true" />
<property name="hibernate.cache.use_query_cache" value="true" />
</properties>
</persistence-unit>
</persistence>
```

3.4 Validaciones de los Java Bean

El API Java para Validación de Java Beans (Bean Validation) proporciona un mecanismo para la validación de datos de la aplicación. Bean Validation define un modelo de **constraints** (restricciones) basado en anotaciones, las mismas que pueden ser colocadas sobre un campo(field), método o una clase de un componente Java Beans, como un CDI Managed Bean o una Entity. La siguiente imagen muestra el ciclo de vida de Bean Validation al ejecutar métodos.



Ciclo de vida de Bean Validation al ejecutar métodos.

Reglas:

1. Para los atributos de una Entidad de persistencia que requieran validación se debe usar el API de Bean Validation. Los atributos para validar no necesariamente pueden ser datos de ingreso o salida de una página web.
2. Para los atributos, o parámetros de métodos no estáticos de un componente web CDI *Managed Bean* que requieran validación se debe usar el API de Bean Validation (ver sección: *Validación de métodos en CDI Managed Beans*). Solo validar en el CDI *Managed Bean* con Bean Validation los campos o propiedades que forman parte de la página web como datos de ingreso.
3. En los parámetros de los métodos de servicios web tipo REST que requieran validación se debe usar el API de Bean Validation.

Utilidad: El uso del API de Bean Validation permite tener un único esquema de validación de Java Beans que se propague por todas las capas de la aplicación.

3.4.1 Constraints de Bean Validation

Bean Validation provee un conjunto de *constraints* (restricciones), las cuales son anotaciones que pueden ser utilizadas en los atributos, o parámetros de métodos de un CDI Managed Bean o servicio web tipo REST. Los constraints que se pueden utilizar son los siguientes (paquete `jakarta.validation.constraints`):

Constraint	Descripción	Ejemplo
@DecimalMax	El campo tiene que ser un número cuyo valor sea menor o igual a un máximo especificado.	@DecimalMax (value = "7000.00", message = "{invalid.salaryDecimalMax}") private BigDecimal sueldo;
@DecimalMin	El campo tiene que ser un número cuyo valor sea mayor o igual a un mínimo especificado.	@DecimalMin (value = "550.00", message = "{invalid.salaryDecimalMin}") private BigDecimal sueldo;
@Digits	El campo tiene que ser un número cuyo valor se encuentre en un rango definido. El elemento integer especifica el máximo de dígitos enteros para el número, y el	@Digits (integer=4, fraction=2, message = "{invalid.discountNumberRange}") private BigDecimal descuento;

	elemento fracción especifica el máximo de dígitos fraccionarios para el número	
@Future	El campo tiene que ser una fecha en un futuro.	@Future (message = "{invalid.dateFuture}") private Date eventDate;
@Max	El campo tiene que ser un número cuyo valor sea menor o igual a un máximo especificado.	@Max (value = 120, message = "{invalid.ageMax}") private int edad;
@Min	El campo tiene que ser un número cuyo valor sea mayor o igual a un mínimo especificado.	@Min (value = 18, message = "{invalid.ageMin}") private int edad;
@NotNull	El campo no puede tener valor null.	@NotNull (message = "{invalid.firsName}") private String firstName;
@Past	El campo tiene que ser una fecha en el pasado.	@Temporal(jakarta.persistence.TemporalType.DATE) @Past (message = "{invalid.datepast}") private Date birthday;
@Pattern	La cadena tiene que coincidir con el patrón de expresión regular especificado.	@Pattern (regexp = "[a-z0-9!#\$%&'*/+=?^_`{ }~-(?:\\. "+ "[a-z0-9!#\$%&'*/+=?^_`{ }~-(?:\\. "+ "(?:[a-z0-9](?:[a-z0-9]*[a-z0-9])?\\.)+[a-z0-9]" + "(?:[a-z0-9-]*[a-z0-9])?", message = "{invalid.email}") private String email;
@Size	El tamaño del campo tiene que estar dentro de un límite especificado (incluyente).	@Size (min=2, max=10, message = "{invalid.sizeName}") private String firstName; @Size (min=2, message = "{invalid.sizeLastName}") private String lastName;
@Valid	Valida un objeto java bean que esta como parámetro en: un método de un componente web CDI Managed Bean o en un recurso Restful.	public String metodo(@NotNull @Valid Contacto contacto){...}

Regla: Todos los campos o propiedades que tengan los constraints (@Size, @Digits, @DecimalMin, @DecimalMax, @Valid) debe ir precedido por el constraint @NotNull.

Ejemplos:

```

@NotNull(message = "{invalid.firstName}")
@Size(min=2, max=10, message = "{invalid.sizeName}")
private String firstName;

@NotNull(message = "{invalid.notNullTotalSalary}")
@Digits(integer=4, fraction=2, message = "{invalid.totalSalaryNumberRange}")
private BigDecimal totalSueldo;

@POST
@Path("/ingresar")
@Produces(MediaType.APPLICATION_JSON)
public String ingresar(@NotNull(message = "{contact.object.notNull}") @Valid Contacto contacto){
    return "EXITO";
}

```

Importante: Si dentro de una entidad de persistencia se pone en algún campo (field) o atributo validaciones de la anotación **@Column** del paquete `jakarta.persistence` juntamente con un constraint de Bean Validation, la validación que tiene prioridad en ejecutarse es la de Bean Validation. Esto debido a que la validación de **@Column** que es de JPA se ejecuta en la capa de persistencia, mientras que los constraint de Bean Validation se ejecutan en la capa de presentación.

3.4.2 Validación de métodos en CDI Managed Beans

Los constraints de Bean Validation deben ser colocados en los parámetros de los métodos no estáticos que requieran ser validados antes de ejecutar la lógica de negocio, para lo cual tomar en cuenta las siguientes reglas:

1. Deben llevar constraints de Bean Validation solo los parámetros de métodos de los componentes web CDI Managed Beans que requieran validación transversal a la lógica de negocio, es decir no validar con Bean Validation los parámetros que NO tengan que ver nada con la lógica de negocio.
2. Deben llevar constraints de Bean Validation solo los parámetros de métodos de los componentes web CDI Managed Beans, y que además el método va a ser invocado solo entre managed beans. En el siguiente ejemplo, si el método `calcularTotalSueldo()` es invocado desde el propio managed bean `SalaryController` los constraints (**@NotNull** y **@Digits**) no serán tomados en cuenta:

```

@Named
@RequestScoped
public class SalaryController implements Serializable {

    @Inject ContactController contactoController;
    public SalaryController() {}

    public BigDecimal calcularTotalSueldo(@NotNull @Digits(integer=3, fraction=2, message =
    "{invalid.reserveFundsNumberRange}") BigDecimal fondosReserva){
        BigDecimal totalSueldo =contactoController.getSueldo().subtract(contactoController.getDescuento());
        totalSueldo = totalSueldo.add(fondosReserva);
        return totalSueldo;
    }
}

```

```
}
```

3. No se deben usar constraints de Bean Validation en los métodos que se está sobre escribiendo de una clase heredada en un CDI Managed bean, como el siguiente ejemplo erróneo:

```
public class Persona {
    ...
    public void setEmail(String email) { ... }
}
public class Empleado extends Persona {
    ...
    @Override
    public void setEmail(@NotNull String email) { ... }
}
```

4. Cuando se invoque un método de un componente web CDI Managed bean que contenga en los parámetros de entrada constraints de Bean Validation, se debe capturar `jakarta.validation.ConstraintViolationException` poniendo en el log solo el mensaje de la excepción, de esta manera ayudará a identificar en que parámetro se está violando el constraint y se debe retornar al usuario final un mensaje personalizado entendible.

Ejemplo:

```
@Named
@SessionScoped
public class ContactController implements Serializable {

    @Inject SalaryController salaryController; //componente web CDI Managed Bean
    public String create() {
        try {
            BigDecimal totalSueldo = salaryController.calcularTotalSueldo(fondosReserva);
            current.setTotalSueldo(totalSueldo);
            getFacade().create(current);
            return "";
        } catch (ConstraintViolationException e) {
            logger.error(e.getMessage());
            //Mensaje que retorna al usuario final
            Util.addErrorMessage(e, ResourceBundle.getBundle("/messages").getString("PersistenceErrorOccured"));
            return "";
        } catch (Exception e) {
            logger.error(e);
            return "";
        }
    }
}
```

3.4.3 Creación de constraints personalizados

Regla: Cuando exista el requerimiento de validar (validación personalizada) algún campo (field) con el API de Bean Validation, y que con los constraints dados por el API no es posible hacer tal validación, entonces se debe hacer un constraint personalizado.

Utilidad: La creación y uso de constraints personalizados permite esconder cierta validación para un campo, en donde el uso de esa validación se reduce en añadir una simple anotación en el campo que requiera ese constraint personalizado.

Ejemplo:

Se tiene un campo (field) tipo 'String' llamado '**email**' que forma parte del java bean llamado '**Contacto**', para lo cual se requiere hacer un constraint personalizado para validar el valor del correo electrónico del campo 'email'. Para implementar el constraint personalizado se debe hacer lo siguiente:

Crear un cualificador o anotación personalizada que representa el constraint personalizado, para este caso se crea la interface '**Email**' anotada con **@Constraint** del paquete jakarta.validation

```
@Documented
@Target({ ElementType.ANNOTATION_TYPE,
ElementType.METHOD,
ElementType.FIELD,
ElementType.PARAMETER })
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = EmailValidator.class)
public @interface Email {

    String message() default "{invalid.email}";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

Implementar la interface 'jakarta.validation.**ConstraintValidator**', en cuya implementación se pone la lógica de la validación personalizada del constraint.

```
public class EmailValidator implements ConstraintValidator<Email, String> {
    private static final String PATTERN_EMAIL= "[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\\.([a-z0-9!#$%&'*/+=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?)?";
    public void initialize(Email constraintAnnotation) {
        ....
    }
    public boolean isValid(String value, ConstraintValidatorContext context) {
        Pattern patron = Pattern.compile(PATTERN_EMAIL);
        Matcher m = patron.matcher(value);
        if (m.find()) {
            return true;
        }
        return false;
    }
}
```

Finalmente, para utilizar al constraint personalizado, solo se debe usar el nombre de la anotación creada anteriormente en el campo 'email', ejemplo:

```
@Email
private String email;
```

3.4.4 Validación de servicios web REST

Regla: Deben llevar constraints de Bean Validation solo los parámetros de los métodos o recursos de un servicio web REST que requieran ser validados antes de ejecutar la lógica en el recurso.

Utilidad: JAX-RS soporta Bean Validation para validar los recursos REST, con lo cual se puede: añadir constraints a los parámetros de los métodos o recursos y asegurar que un objeto java bean sea validado cuando éste es pasado como parámetro en un recurso.

Ejemplo:

Se tiene el servicio web REST llamado '*Persons*', el cual contiene el recurso llamado '*create*' y se requiere validar con Bean Validation sus parámetros 'id' y 'name':

```
@Path("persons")
public class Persons {
    @POST
    @Path("create")
    @Produces(MediaType.APPLICATION_JSON)
    public Response createPerson(
        @FormParam("id")
        @NotNull(message = "{person.id.notnull}")
        @Pattern(regexp = "[0-9]+", message = "{person.id.pattern}")
        String id,
        @FormParam("name")
        @Tamano String name) /*@Tamano es un constraint de Bean Validation personalizado*/ {
        Person person = new Person();
        person.setId(Integer.valueOf(id));
        person.setName(name);
        persons.put(id, person);
        return Response.status(Response.Status.CREATED).entity(person).build();
    }
    ...
}
```

3.4.5 Mapeo de excepciones de Bean Validation con JAX-RS

Si una `jakarta.validation.ValidationException` o alguna subclase de `ValidationException` excepto `ConstraintValidationException` es lanzada, el JAX-RS en tiempo de ejecución responderá la petición del cliente con un error **500** (Internal Server Error) HTTP. Por otro lado, si un `ConstraintValidationException` es lanzado, el JAX-RS en tiempo de ejecución responderá la petición del cliente con uno de los siguientes errores: **500** (Internal Server Error) o **400** (Bad Request). Este comportamiento puede ser personalizado con lo siguiente:

Regla: Para enviar como respuesta (*personalizada*) al cliente los mensajes dados por Bean Validation al hacer una petición a algún recurso REST, se debe implementar la interface `jakarta.ws.rs.ext.ExceptionMapper` que mapea excepciones del tipo `ValidationException`. La implementación de `ExceptionMapper` es lanzada cuando alguna validación con Bean Validation falla.

Ejemplo:

Se requiere implementar la interface `ExceptionHandler` para capturar los errores lanzados por Bean Validation al ejecutar métodos o recursos REST.

```
@jakarta.ws.rs.ext.Provider
public class ValidationExceptionHandler implements ExceptionHandler<ValidationException> {

    public Response toResponse(final ValidationException exception) {
        if (exception instanceof ConstraintViolationException) {
            LOGGER.log(Level.FINER, "Following ConstraintViolations has been encountered.", exception);
            final ConstraintViolationException cve = (ConstraintViolationException) exception;
            final Response.ResponseBuilder response = Response.status(getStatus(cve));

            final List<Variant> variants =
                Variant.mediaTypes(MediaType.APPLICATION_XML_TYPE, MediaType.APPLICATION_JSON_TYPE).
                build();
            final Variant variant = request.get().selectVariant(variants);
            if (variant != null) {
                response.type(variant.getMediaType());
            }
            response.entity(new
                GenericEntity<List<ValidationError>>(getEntity(cve.getConstraintViolations()), new
                GenericType<List<ValidationError>>().getType()));

            return response.build();
        } else {

            LOGGER.log(Level.WARNING, "Unexpected Bean Validation problem.", exception);
            return Response.serverError().entity(exception.toString()).build();
        }
    }
}
```

3.4.6 Personalización de mensajes de constraints de Bean Validation

Bean Validation incluye un recurso para mensajes por defecto correspondiente a los constraints.

Regla: Todos los mensajes de los constraints de Bean Validation que se utilicen en la aplicación, deben tener un mensaje descriptivo personalizado. Para esto se debe crear el archivo de propiedades llamado **messages_es.properties** y ubicarlo en la aplicación en la parte correspondiente a la capa de presentación en la ruta **/src/main/resources**, en este archivo se deben poner los mensajes como clave-valor, ejemplo:

```
invalid.email=No es una dirección de correo electrónico válida.
invalid.datepast=Fecha de nacimiento debe ser del pasado.
invalid.lastName=Apellido es obligatorio.
invalid.sizeName=El tamaño del nombre es incorrecto min2, max10.
```

```
@Entity
@Table(name="CONTACTO")
public class Contact implements Serializable {
```

```
@NotNull(message = "{invalid.lastName}")
@Size(min=2, message = "{invalid.sizeLastName}")
private String lastName;
...
}
```

Importante: El archivo **messages_es.properties** creado, es para localización de mensajes en español indistintamente del país; pero si se requiere crear un archivo de mensajes con localización diferente se debe concatenar el **prefijo del locale del idioma** al nombre del archivo base (MensajesDeValidacion.properties); ejemplo de nombre de archivo de mensajes en idioma inglés indistintamente del país **messages_en.properties**:

```
invalid.email=Correo inválido.
invalid.datepast=La fecha debe ser anterior.
```

3.5 Mensajería

3.5.1 Reglas de uso

Regla: Las soluciones que requieran comunicación asíncrona basada en mensajería deben implementarse utilizando la especificación Jakarta Messaging soportada por la versión de la Plataforma Jakarta EE.

Utilidad: Implementar comunicación asíncrona desacoplada, interoperable y portable entre componentes y sistemas empresariales, favoreciendo escalabilidad, tolerancia a fallos, integración distribuida y compatibilidad con la evolución de la Plataforma Jakarta EE.

Regla: Se deben usar Colas/Tópicos definidos al momento del despliegue siempre y cuando los productores/consumidores estén definidos dentro del mismo despliegue o se establezca un mecanismo mediante el cual los productores/consumidores puedan ser notificados que el recurso Cola/Tópico está disponible.

Utilidad: Evitar errores por búsqueda al no encontrarse los recursos Colas/Tópicos disponibles.

Consideraciones:

- Los mensajes que se guarden en los recursos Colas/Tópicos definidos al momento del despliegue son persistentes y únicamente pueden ser accedidos si la aplicación está desplegada.
- Los recursos Colas/Tópicos definidos al momento del despliegue pueden ser accedidos remotamente.

3.5.2 Producir y consumir

Regla: Los mensajes provenientes de colas o tópicos Jakarta Messaging (JMS) que requieran procesamiento asíncrono administrado por el contenedor deben consumirse mediante Message-Driven Beans (MDB).

Utilidad: Los MDB constituyen el mecanismo estándar para la recepción y procesamiento de mensajes JMS en aplicaciones Jakarta EE.

Regla: Para producir mensajes de una cola/tópico en un módulo cliente se debe usar el API provisto por JMS, para acceder a los recursos se debe usar la interface: *jakarta.jms.Destination* y las interfaces de JMS *jakarta.jms.JMSContext*, *jakarta.jms.JMSProducer*.

Utilidad: Portabilidad del código a través de diferentes proveedores.

Regla: Las aplicaciones Jakarta EE no deberán crear ni administrar hilos directamente mediante las clases `java.lang.Thread`, `java.util.concurrent.ExecutorService`, `java.util.concurrent.ThreadPoolExecutor` u otros mecanismos equivalentes de Java SE. Para procesamiento asíncrono o concurrente, se debe utilizar exclusivamente los mecanismos administrados por el contenedor, tales como `@Asynchronous` (Enterprise Beans), Jakarta Concurrency (`ManagedExecutorService`, `ManagedScheduledExecutorService`) o Jakarta Messaging (JMS), según el escenario.

Utilidad: Garantizar que la ejecución concurrente permanezca bajo el control del contenedor Jakarta EE, preservando correctamente los contextos de transacción, seguridad, CDI y persistencia, además de mejorar la estabilidad, escalabilidad y portabilidad de las aplicaciones empresariales.

Regla: Las aplicaciones que requieran balancear el procesamiento de mensajes publicados en un tópico Jakarta Messaging entre múltiples instancias consumidoras deben implementar suscripciones compartidas (*Shared Subscriptions*) utilizando los mecanismos estándar definidos por la especificación Jakarta Messaging.

Utilidad: Garantizar una distribución eficiente de carga, optimiza el uso de recursos de infraestructura y facilita la escalabilidad horizontal de los componentes consumidores.

3.5.3 Procesamiento por lotes

El detalle referente a este tema se lo puede ver en el documento llamado “*Estándares de Desarrollo de Aplicaciones Batch*”.

4 Manejo de logs y excepciones

4.1 Manejo de logs

El registro de log es un mecanismo que permite hacer el seguimiento del comportamiento en tiempo de ejecución de una aplicación, proporcionando información importante para los desarrolladores en el proceso de identificación o depuración de algún error de la aplicación y al operador para el proceso administrativo y de mantenimiento de las aplicaciones en producción. Con esto se asegura que la aplicación realice las operaciones que se esperaba. En algunos casos, estos registros también se utilizan para fines de auditoría.

4.1.1 Configuración a nivel de Jakarta EE

Regla: Todas las aplicaciones Jakarta EE deben implementar el registro (log) de eventos, errores, etc., utilizando el API JBoss Logging como mecanismo estándar de logging.

Utilidad: Estandarizar la generación de registros (logs) de aplicación, facilitar el monitoreo y diagnóstico de errores, y garantizar la integración consistente con la infraestructura de logging y observabilidad de la plataforma Jakarta EE.

Consideraciones:

1. JBoss Logging requiere un API de acceso por lo que debe incluirse en el archivo pom.xml del proyecto principal la siguiente dependencia:

```
<!-- uso del API logging -->
<dependency>
<groupId>org.jboss.logging</groupId>
<artifactId>jboss-logging</artifactId>
<scope>provided</scope>
</dependency>
</dependency>
```

La implementación de los logs debe ser desarrollada con la siguiente importación:

```
import org.jboss.logging.Logger;
```

Regla: Para las configuraciones de Logs a nivel de servidor de aplicaciones standalone se debe seguir las reglas correspondientes descritas en el “Estándar de Configuración de Servidores de Aplicaciones JBoss EAP”.

4.1.2 Implementación general del log

Regla: Los componentes de la aplicación deben implementar el registro (log) de eventos, errores, etc. utilizando el API estándar de JBoss Logging, mediante la factoría `Logger.getLogger()`.

```
private static final Logger LOG = Logger.getLogger(NOMBRE_CLASE.class);
```

Utilidad: Permite estandarizar la generación de registros en toda la aplicación, facilitando la identificación del origen de los mensajes, mejorando la trazabilidad operativa y garantizando una integración consistente con la infraestructura de logging de la plataforma Jakarta EE.

Regla: El registro (log) de excepciones debe realizarse en el punto más alto de la pila de ejecución donde la excepción sea finalmente tratada o controlada, no se debe registrar repetidamente una misma excepción en múltiples capas de la aplicación, a fin de prevenir duplicidad de registros (logs) y pérdida de claridad en el diagnóstico de algún evento o error en la aplicación.

Regla: Para el registro (log) debe implementarse utilizando los métodos de JBoss Logging (`tracef`, `debugf`, `infof`, `warnf`, `errorf`), evitando la concatenación de cadenas para la construcción de mensajes de log.

Utilidad: Mejora la legibilidad del código, reducir procesamiento innecesario cuando determinados niveles de logging están deshabilitados y mantener uniformidad en la implementación de registros (logs) de aplicación.

Regla: Se debe hacer previamente la verificación de los niveles de log `INFO`, `DEBUG` y `TRACE` mediante los métodos `isInfoEnabled`, `isDebugEnabled()` o `isTraceEnabled()` respectivamente, cuando únicamente el mensaje de log tenga operaciones que impliquen procesamiento costoso, tales como: conversión de objetos, serialización, concatenación o invocación de métodos, etc.

Utilidad: Comprueba niveles de logs activos antes de registrarlo para evitar consumo innecesario de CPU, memoria y problemas de rendimiento.

Ejemplo:

```
if (LOG.isDebugEnabled()) { // operación de procesamiento costosa: objetoMapper.writeValueAsString()

    LOG.debugf("Respuesta completa: %s", objetoMapper.writeValueAsString(respuesta));
}
```

Nota: Para el registro (log) de mensaje simples se puede utilizar directamente los métodos debugf o infof del JBoss Logging.

Ejemplos:

```
LOG.debugf("Usuario %s procesado", usuariold);

LOG.infof("Proceso iniciado.");
```

Regla: No utilizar: system.out, system.err, printStackTrace. Como mecanismo para el control, depuración y mantenimiento de la aplicación se debe utilizar el registro de log con su nivel respectivo.

Utilidad: Los métodos: system.out, system.err, printStackTrace, son muy costosos y generan problemas de rendimiento y memoria a la aplicación.

Ejemplo:

INCORRECTO:

```
} catch (AuditoriaExcepcion ex) {
    ex.printStackTrace();
    return null;
}
```

CORRECTO:

```
} catch (AuditoriaExcepcion ex) {
    LOG.errorf(ex, "ERROR AL OBTENER INFORMACION DEL USUARIO AUTENTICADO");
    return null;
}
```

Regla: Cuando se registre (log) una excepción mediante JBoss Logging, el objeto Throwable debe proporcionarse como el primer argumento de los métodos de registro que admitan una excepción, seguido del mensaje o formato y, cuando corresponda, de los parámetros utilizados para el formateo del mensaje.

Utilidad: Evita que la excepción sea tratada accidentalmente como un parámetro del mensaje y garantiza que el stack trace y causa sean gestionados correctamente por JBoss Logging.

Ejemplo:

```
LOG.errorf(ex, "Error al obtener información del usuario autenticado.");
LOG.errorf(ex, "Error procesando solicitud %s ", solicitudId);
LOG.warnf(ex, "No fue posible consultar el cliente %s ", clienteId);
```

Regla: Las pruebas unitarias no deben implementar logs.

Consideraciones:

- Implementar el registro de logs solo en casos necesarios.
- El registro de un log no debe causar excepciones ejemplo:

Al ejecutar esta sentencia hay que asegurarse que el objeto politicaAuditoria no sea nulo:

```
LOG.debugf("PROCESANDO POLITICA %s ", politicaAuditoria.getCodigoPolitica());
```

Un objeto como usuariosLista, manipulado incorrectamente en el log puede causar excepciones como OutOfMemory, LazyInitializationExceptions, etc:

```
LOG.debugf("PROCESANDO USUARIOS %s ", usuariosLista);
```

- La concatenación de Strings es costosa, teniendo en cuenta que en Java los Strings son inmutables y se crea un pool de Strings que se almacena en la memoria dinámica.
- Asegurarse que el log brinde la información suficiente y consistente.

INCORRECTO:

```
LOG.debugf("INICIO");
```

CORRECTO:

```
LOG.debugf("INICIO DEL PROCESO ASINCRONO DE AUDITORIA %s ", politicaAuditoria);
```

4.1.3 Implementación de mensajes logs

Los mensajes logs representan la implementación de interfaces que definen métodos genéricos. Estos proporcionan un mayor nivel de identificación y mayor abstracción proporcionando un catálogo personalizado de mensajes en el log general.

Regla: Los mensajes logs deben alinearse a las directrices de configuración e implementación general del log.

Regla: Todos los mensajes logs deben poseer un código de proyecto general de máximo 9 caracteres según el contexto de negocio y un identificador de rango según su nivel.

Nivel	Identificador	Ejemplo
DEBUG	100	101, 102, ... 199
INFO	200	201, 202, ... 299
WARN	300	301, 302, ... 399
ERROR	400	401, 402, ... 499
FATAL	500	501, 502, ... 599

4.1.4 Implementación del log con MDC

El **Mapped Diagnostic Context** (MDC) o Contexto de Diagnóstico Mapeado es una funcionalidad de los frameworks de logging que permite asociar información contextual a un hilo de ejecución para que dicha información se incluya automáticamente en todos los registros (logs) generados durante transacción.

En JBoss EAP 8.x, JBoss Logging se integra con la infraestructura de logging del servidor, que soporta MDC mediante clases como “org.jboss.logging.MDC”.

Regla: Las aplicaciones deben utilizar un Contexto de Diagnóstico Mapeado (MDC) para incorporar en los mensajes de registro (log) con partes de información de correlación y trazabilidad que podrían no estar disponibles en el ámbito en el que el registro se produce. Por ejemplo:

- En aplicaciones distribuidas o que tienen múltiples clientes simultáneamente
- En una trazabilidad donde se necesite relacionar el log de la aplicación con la pista de auditoria donde el dato de la correlación es el nombre de host, nombre de usuario y/o su dirección IP.

Utilidad: Permite correlacionar registros generados por múltiples componentes y procesos concurrentes, facilitando la trazabilidad operativa, el diagnóstico de incidentes y la observabilidad de aplicaciones empresariales.

Regla: La implementación de MDC debe estar desacoplada de la lógica de negocio por medio del desarrollo de interceptores CDI. Todas las partes de información que se agregan al log al inicio de cada proceso deben ser removidas o limpiadas del MDC al finalizar su ejecución.

4.1.5 Niveles de un log

Regla: A continuación, se definen las reglas para usar niveles de log:

- **DEBUG:** Usar este nivel siempre que se esté investigando un error específico.
- **INFO:** Usar este nivel para emitir información general como: inicio, apagado, versión y configuración de una aplicación. También para emitir información acerca de decisiones importantes en el código o cambios en la configuración.
- **WARN:** Usar este nivel para emitir una advertencia o información que no representan un problema por sí mismo.
- **ERROR:** Usar este nivel para identificar un problema y se tienen que tomar acción sobre esto.
- **FATAL:** Usar este nivel si la aplicación presenta un problema con un estado irreparable como por ejemplo excepciones lanzadas por la JVM (OutOfMemoryError), cuando esté relacionado a un apagado del servidor o a matar la sesión.

Regla: Para la configuración de los niveles de log en los ambientes productivos y no productivos se debe aplicar las reglas del “Estándar de Configuración de Servidores de Aplicaciones JBoss EAP” vigente, de la sección “Logs”.

Consideración: Si se requiere un nivel inferior a “ERROR” en el ambiente de producción se deberá solicitar por un periodo de tiempo específico al departamento de Infraestructura y

Operaciones mediante requerimiento indicando la categoría o log, el tiempo (menor a 4 horas) y el nivel de log requerido.

4.1.6 Correlación de la información del log con las pistas de auditoría

El mecanismo que registra pistas de auditoría para cada uno de los sistemas define en su estructura una información básica obligatoria. Entre esa información básica está el nombre del usuario, la dirección IP, el módulo de negocio, el tipo de auditoría, la fecha, la hora, la política, etc.

Ejemplo:

```
{
  "usuario": "1710034065001",
  "hostname": "56",
  "ipCliente": "10.21.178.199",
  "codigoAplicacion": "DVI",
  "codigoModulo": "DVI",
  "ambiente": "INTRANET",
  "codigoAplicacionOrigen": "DVI",
  "fechaHora": "29/05/2018 12:00:12,793",
  "tipoAuditoria": "FUNCIONALIDAD_CRITICA"
}
```

Regla: Los atributos que proporcionan una correlación con la pista de auditoría son: nombre de usuario, nombre de host, dirección IP, aplicación, módulo, fecha y hora, y tipo de auditoría. Proporcionar la correlación de estos atributos mediante MDC.

4.1.7 Herramienta de análisis de logs

Regla: Para la visualización y análisis de logs usar las herramientas que se enlistan en [Anexo 2: Tecnologías/Herramientas a utilizar](#).

4.2 Manejo de Excepciones

4.2.1 Implementación de excepciones:

Regla: En los modelos y DAOS no se debe controlar ni lanzar la excepción. Las excepciones de tipo no controladas que ocurran en la capa de acceso a datos (DAO) deben ser capturadas en la capa de servicios, transformadas en una excepción de tipo aplicación y lanzadas a otra capa superior.

Regla: Las excepciones controladas deben ser capturadas en el método en el que se producen (no se consideran clases de modelos y DAOS), transformadas en las excepciones de aplicación adecuadas y enviadas hacia las capas superiores.

Regla: Las excepciones de tipo no controlado que se produzcan en la capa de servicio se considerarán en general errores de la aplicación. Para evitar que su ocurrencia se muestre al usuario se debe utilizar el mecanismo de mensajes de errores web al usuario final.

Regla: En la capa de presentación o servicios web solo debe realizar el control de las excepciones, en este nivel no deben existir métodos que vuelvan a lanzar las excepciones.

Regla: No se debe capturar la excepción genérica (`java.lang.Exception`) sino únicamente cuando APIs de terceros lo obligan.

Regla: En la capa de vista/controlador se deben capturar tanto las excepciones de tipo aplicación que provienen de las capas inferiores como las que se produzcan en esta capa. Las excepciones serán capturadas en el backing bean dentro de cada uno de los métodos donde se ejecuta el acceso a la capa de servicio.

Regla: La aplicación deberá gestionar los mensajes orientados a los usuarios finales y a los administradores técnicos. Los primeros son los que se muestren en las páginas de la aplicación y los segundos los que se registren en el log del sistema.

Regla: Las clases que administren recursos externos o no administrados por el contenedor, tales como archivos, streams, sockets, conexiones o recursos equivalente, deben implementar `java.lang.AutoCloseable` (o `java.io.Closeable` cuando corresponda) para permitir su liberación automática mediante la estructura **try-with-resources**.

Utilidad: Garantizar el cierre seguro y automático de recursos del sistema, reduciendo fugas de memoria, agotamiento de conexiones, bloqueos de archivos y problemas de estabilidad o rendimiento en aplicaciones Jakarta EE.

Regla: Usar catch múltiple, para la agrupación de tipos de Excepciones mediante el signo '|'

Ejemplo:

```
try {  
    // algún código que lance algunas de las tres excepciones.  
} catch(SQLException | IOException e) {  
    LOG.errorf(e);  
} catch(Exception e) {  
    LOG.errorf(e);  
}
```

Regla: Se debe utilizar **enmascaramiento de excepciones** como técnica en el manejo de excepciones.

Utilidad:

- Prevenir que lleguen hasta la tope del *call stack* (*pila de llamadas*) todas las posibles excepciones del sistema.
- Se prefiere que los componentes de más alto nivel no conozcan mucho de la implementación de los componentes de más bajo nivel.
- Siempre se debe poner la causa de la excepción en el enmascaramiento.
- Se prefiere el enmascaramiento cuando se requiera agregar información adicional a la excepción.

Ejemplo:

```
try{  
    ...  
    appService.guardarEntidad(contribuyente);  
} catch (SQLException sqlException) {
```

```
        throw new ApplicationException("Error guardando la Entidad:", sqlException);
    }
```

Regla: En el bloque de código donde se realiza el manejo de excepciones debe ser “**A prueba de errores**”. El código de limpieza y recuperación debe ejecutarse de forma segura y no debe provocar el enmascaramiento de la excepción que originó el fallo.

Utilidad: Permite que durante el análisis de errores se tengan los datos correctos, y no se enmascare la excepción original y su stack trace, evitando que una excepción generada durante el manejo del error se convierta en la última excepción observada y oculte la causa real del problema.

Ejemplo:

- Manera incorrecta, variable *input* puede ser *null*.

```
//Incorrecto, puede enmascarar la excepción original:

InputStream input = null;
    try {
        InputStream input = new FileInputStream("archivo.txt");
        // hacer algo con el stream
    } catch (IOException e) {
        throw new WrapperException(e);
    } finally {
        try {
            input.close();
        } catch (IOException e) {
            throw new WrapperException(e);
        }
    }

//Correcto, garantiza automáticamente el cierre de input:

try (InputStream input =
    new FileInputStream("archivo.txt")) {

    // Procesar el stream

} catch (IOException e) {
    throw new WrapperException(e);
}
```

Consideración: La última excepción lanzada en un bloque *try/catch/finally* es la excepción que se propaga hacia arriba en la pila de llamadas, por lo tanto, todas las excepciones anteriores desaparecen.

4.2.2 Excepciones en el contexto de componentes de negocio CDI

Regla: Las excepciones de aplicación que representen validaciones, restricciones funcionales o condiciones esperadas del negocio deben propagarse al cliente conservando su significado funcional y la información relevante del error, y cuando el comportamiento por defecto de rollback no sea suficiente deberá definirse explícitamente utilizando los atributos `rollbackOn` o `dontRollbackOn` en la anotación `@Transactional`.

Utilidad: Diferenciar los errores funcionales de los errores técnicos y permite controlar explícitamente el rollback de las transacciones administradas por Jakarta Transactions.

Nota: Definición de una excepción de aplicación

- Las excepciones de aplicación representan condiciones funcionales esperadas del dominio de negocio.
- Pueden declararse en métodos de servicios CDI, recursos REST, componentes Batch, consumidores o productores de mensajería y demás componentes administrados por CDI.
- Pueden implementarse como excepciones chequeadas (Exception) o no chequeadas (RuntimeException), de acuerdo con el modelo de excepciones definido por la aplicación.

Regla: Las excepciones técnicas, errores de infraestructura o fallos inesperados no deben transformarse en excepciones de negocio. Cuando un servicio CDI no pueda recuperarse de una condición técnica, deberá propagar la excepción para que el interceptor transaccional de Jakarta Transactions gestione el rollback conforme a la configuración transaccional definida.

Utilidad: Garantiza la integridad transaccional y evita ocultar errores técnicos que deben ser administrados por la plataforma.

Regla: Para el manejo de excepciones en servicios CDI se deberá considerar lo siguiente:

- Las excepciones técnicas o errores no recuperables no deben capturarse innecesariamente; deben propagarse para permitir el manejo adecuado del contexto transaccional.
- Las excepciones de infraestructura, persistencia o integración deberán encapsularse únicamente cuando sea necesario desacoplar dependencias tecnológicas o agregar información de contexto, preservando siempre la causa original de la excepción.
- Las condiciones inesperadas, errores de persistencia, fallos de conectividad y demás problemas no funcionales deben tratarse como excepciones técnicas y no como excepciones de negocio.
- No se debe utilizar Exception, RuntimeException o Throwable como excepciones genéricas de negocio.

Utilidad: Favorece un modelo consistente de manejo de errores, mejora la mantenibilidad de los servicios y preserva el comportamiento transaccional definido por Jakarta Transactions.

4.2.3 Excepciones en el contexto EJB

Regla: Las excepciones de aplicación que representen reglas funcionales o condiciones esperadas del negocio deben propagarse al cliente conservando su significado funcional y la

información relevante del error, y puede configurarse para provocar o no rollback transaccional cuando el comportamiento por defecto no sea suficiente.

Utilidad: Diferenciar errores funcionales de fallos técnicos, garantizando un manejo consistente de excepciones y un control adecuado del comportamiento transaccional en aplicaciones empresariales Jakarta EE.

Nota: Definición de una excepción de aplicación

- Las excepciones de aplicación deben utilizarse para representar validaciones, restricciones funcionales o errores esperados del dominio de negocio.
- Una excepción de aplicación puede declararse en la cláusula throws de métodos expuestos mediante interfaces de negocio EJB, no-interface view, endpoints de servicios web, listeners de MDB o componentes equivalentes.
- Una excepción de aplicación puede ser una subclase (directa o indirecta) tanto de *java.lang.Exception* (“excepción chequeada”), como una subclase de *java.lang.RuntimeException* (“excepción no chequeada”).

Regla: Las excepciones de la JVM, errores inesperados de infraestructura o fallos del sistema no deben ser tratados como excepciones de negocio. En componentes EJB, este tipo de errores debe propagarse al contenedor para que éste gestione adecuadamente el rollback transaccional, la invalidación del componente y el manejo de la excepción conforme a la especificación Jakarta Enterprise Beans.

Utilidad: Garantiza que el contenedor EJB administre correctamente los errores del sistema, preservando la integridad transaccional y la estabilidad de la aplicación.

Regla: Cuando una excepción de aplicación requiera provocar rollback transaccional y dicho comportamiento no se encuentre configurado mediante @ApplicationException (rollback = true) o configuración transaccional equivalente, el componente EJB deberá marcar explícitamente la transacción para rollback utilizando EJBContext.setRollbackOnly() antes de propagar la excepción.

Utilidad: Garantiza la consistencia transaccional de la operación y evita commits parciales ante errores funcionales que invaliden el proceso de negocio.

Regla: Para el manejo de excepciones de la aplicación en componentes EJB se deberá considerar lo siguiente:

- Las excepciones técnicas o errores no recuperables no deben capturarse innecesariamente dentro del componente EJB; deben propagarse al contenedor para su administración.
- Cuando un componente EJB detecte una excepción chequeada de infraestructura o una condición técnica en la cual no pueda recuperarse, el método del componente EJB debe lanzar un *jakarta.ejb.EJBException* que encapsule la excepción original.
- Las condiciones inesperadas, fallos de infraestructura, errores de persistencia o problemas no funcionales deben tratarse como excepciones del sistema y no como excepciones de negocio.

Nota:

- La excepción *jakarta.ejb.EJBException* es no chequeada, es una subclase de *java.lang.RuntimeException* y no tiene que estar reportada en la firma del método.

Utilidad: Manejo uniforme de errores técnicos bajo el modelo transaccional y operacional administrado por el contenedor Jakarta EE.

4.2.4 Especificaciones de la Implementación

Regla: Se establece un modelo de datos mínimo para el modelamiento de las Excepciones de Aplicación y sirve como la “Descripción de la Excepción” en el modelo de manejo de excepciones propuestas:

- **Identificador de Error:** Un identificador de error único que puede usarse en un sistema de soporte a usuarios.
- **Mensaje:** Un mensaje claro indicando cual es el error que ha ocurrido.
- **Tipo:** Debe ser uno de: Error Interno, Error de servicio o Error de Cliente.
- **Severidad:** Debe incluir un nivel de severidad
- **Identificador del Usuario**
- **Fuente.** - Clase en que se lanzó la excepción, si es posible la fuente debe tener información de la aplicación y módulo
- **Código de error:** Un código que debe estar catalogado en el sistema de soporte a usuarios.
- **Explicación del motivo** por el cual se lanzó la excepción, si es posible incluir el código de la instancia de dominio o caso de uso que está afectado
- **Acción del Usuario.** - Mensaje explicativo de la acción que tiene que realizar el usuario si el framework de manejo de excepciones no la puede manejar automáticamente.
- **Parámetros.** - Datos adicionales requeridos para que el manejador de excepciones, aplique una estrategia de tolerancia a fallos.

Utilidad: Proveer mayor detalle y brindar una opción de primera mano para la corrección de errores.

Ejemplo: Si se realiza el log de las excepciones el siguiente recuadro muestra cómo se debería ver, para hacer esto se debe escribir un patrón de formateo adecuado y utilizar MDC:

```
Error Id : 123456789
Error Code : JMSAMQ2001
Severity: MEDIUM
UserName : hcra040712
Message : JMSAMQ2001: No existe la cola 'testQueue'.
Class : org.gob.Colita(ProcesarColita:35)
Explanation : El MDB trata de abrir una cola que no está registrada.
User Action : Verique que el nombre de la cola está bien escrito.
Parameters: {entityId: 12354, className:ec.gob.Ruc, strategy:retry, retrycount:2}
```

4.2.5 Excepciones en el Contexto WEB: Servlet, JSF, JAX-WS (SOAP), JAX-RS (REST)

Regla: En aplicaciones web que consuman servicios HTTP (REST u otros), las respuestas de error deberán gestionarse mediante los códigos de estado HTTP correspondientes y un formato de respuesta de error uniforme, dependiendo del tipo de contenido se debe aplicar lo siguiente:

- **text/html**.- se puede devolver una página de error genérica que indique el error, la aplicación javascript cliente debe mostrar el error apropiadamente.
- **json**.- Todas las llamadas javascript deben esperar como respuesta un arreglo de objetos (mapa), y el objeto error siempre debe estar incluido, aunque no haya ocurrido ningún error.

Ejemplo:

```
{error:null, persona:{nombre:"Hugo Robayo", ruc:"0919629975003"}}
```

Regla: En servicios web SOAP implementados con Jakarta XML Web Services (JAX-WS), las condiciones de error funcional deberán representarse mediante excepciones específicas del servicio. Dichas excepciones deberán implementarse como excepciones chequeadas (Exception) y serán expuestas a los consumidores mediante elementos SOAP Fault, conforme a la especificación Jakarta XML Web Services.

Utilidad: Permite representar de forma explícita los errores funcionales del servicio, mantener contratos WSDL consistentes y garantizar la interoperabilidad entre plataformas que consumen servicios SOAP.

Regla: Las excepciones funcionales expuestas como parte del contrato del servicio SOAP deberán implementarse preferentemente como excepciones chequeadas (Exception). Las excepciones técnicas o de infraestructura no deberán formar parte del contrato público del servicio.

Regla: Cuando se requiera personalizar el nombre del elemento de falla o la información publicada en el contrato WSDL, se deberá utilizar la anotación jakarta.xml.ws.WebFault.

Utilidad: Permite controlar la representación de las excepciones funcionales en el contrato del servicio sin afectar la implementación del componente.

Regla: Las excepciones técnicas, errores de infraestructura o fallos inesperados no deben exponerse directamente al consumidor del servicio. Dichos errores deberán registrarse en los logs de la aplicación y transformarse en un SOAP Fault apropiado, evitando revelar información sensible de la implementación.

Utilidad: Mejora la seguridad de la aplicación y evita exponer detalles internos de la infraestructura o del código fuente.

Regla: Salvo requerimientos específicos, no se debe modificar manualmente la definición del WSDL, ésta debe ser generada automáticamente por herramientas para dicho fin.

5 Lógica Parametrizable

Regla: Los parámetros utilizados en la lógica de negocio y que se prevea puedan cambiar, deberán ser parametrizados mediante uno de los siguientes métodos:

- ✓ Uso de parámetros generales: esta alternativa se aplica en el caso de requerir configuraciones del tipo nombre valor. Vea también [Catálogos y parámetros](#).

Ejemplo: Parametrización del valor del Impuesto al Valor Agregado.

- ✓ Creación y uso de estructuras de parametrización (tablas propias del sistema): esta alternativa se aplica para funcionalidades de complejidad media por las particularidades de los requerimientos.

Ejemplo: Parametrización de las reglas del Impuesto a la Renta

- ✓ Uso de librería de reglas de negocio (Drools): se utilizará cuando se requiera automatizar decisiones de negocio de complejidad alta que consideren diferentes variables y que pueden ser configuradas como tablas de decisión.

Ejemplo: Notas de crédito para el pago de impuestos que tipo de NC sirve para cada impuesto, para la determinación de la oficina de atención de contribuyente en base a parámetros como jurisdicción el tipo de trámite, el canal de ingreso.

Utilidad: Permitir la flexibilidad para que los sistemas puedan externalizar la lógica de negocio que se considera parametrizable.

Regla: Para el uso de la librería de reglas de negocio (Drools) se requerirá contar con el análisis y aprobación del Arquitecto TI asignado y se deberá colocar la definición correspondiente en la Arquitectura de la aplicación.

Regla: En el caso del uso de la librería de reglas de negocio (Drools), se debe almacenar el archivo de reglas de negocio en la base de datos.

6 Servicios Web

Regla: La seguridad de los servicios web debe cumplir con los lineamientos descritos “Estándar Tecnológico Seguridad de API Web”. (Ver ruta en [Anexo 3: Recursos relacionados](#)). También revise la sección [2 Estándares Generales](#).

Regla: La funcionalidad expuesta mediante servicio web no debe invocar a otro servicio web, se exceptúa la interacción con servicios externos.

Utilidad: Garantizar responsabilidad única y desacoplamiento de la lógica expuesta para interoperabilidad entre dominios de negocio.

Regla: Los formatos de retorno que pueden usarse para intercambio de información mediante servicios de tipo RESTful son JSON, XML y para los casos en los que sea necesario el retorno de archivos se debe usar MIME Type acorde al tipo de archivo.

Regla: El formato de retorno que puede usarse para intercambio de información mediante servicios de tipo SOAP es XML.

Regla: Para revisar los escenarios de uso aprobados para los tipos de servicios revisar la sección [Interoperabilidad](#).

6.1 Servicios Web (SOAP – JAX-WS)

Regla: Para la asignación de clases java representaciones XML se debe utilizar JAXB.

Regla: Un diseño basado en SOAP debe incluir los siguientes elementos.

- Un contrato formal debe establecer la descripción de la interfaz que el servicio web ofrece.
- La arquitectura debe direccionar requerimientos no funcionales complejos (Por ejemplo, transacciones, seguridad, direccionamiento, confianza, coordinación, etc.).
- La arquitectura necesita manejar procesamiento e invocación asíncrona.

6.2 Servicios Web REST (JAX-RS)

Regla: Para acceso a servicios REST se debe utilizar JAX-RS Client API soportado por la plataforma Jakarta EE.

Regla: Se debe utilizar `PathParam` en lugar de `QueryParam` para el manejo de parámetros de los servicios REST.

Ejemplo:

El método `buscarPersona` se ejecuta cuando la petición es igual a `"/persona/nombre/pedro"`. La anotación `@PathParam` que precede al parámetro `"nombre"` del método es encargado de mapear el valor `"pedro"` a la variable `nombre`.

```
@GET
@Path("/persona/nombre/{nombre}")
public Persona buscarPersona(@PathParam("nombre") String nombre) {
    Persona persona = null;
    for (Persona p : listaPersonas)
        if (nombre.equals(p.getNombre())) {
            persona = p;
        }
    return persona;
}
```

6.2.1 Configuración

Regla: Las aplicaciones que tienen un framework de desarrollo javascript en el front-end deben usar servicios REST en su back-end.

Regla: La implementación de servicios web REST debe realizarse con JAX-RS soportado por la plataforma Jakarta EE.

Regla: Se establece que el URI base para una aplicación JAX-RS se localice en el URL a partir de: `"rest"`, configurado en el descriptor de despliegue `web.xml`:

Ejemplo:

```
<servlet>
    <servlet-name>jakarta.ws.rs.core.Application</servlet-name>
</servlet>
<servlet-mapping>
    <servlet-name>jakarta.ws.rs.core.Application</servlet-name>
    <url-pattern>/rest/*</url-pattern>
```

```
</servlet-mapping>
```

Regla: La implementación de las interfaces definidas en JAX-RS para la construcción de servicios web REST se lo debe realizar con el marco de desarrollo RESTEasy.

Utilidad: RESTEasy es un proyecto de JBoss que proporciona diversos marcos de desarrollo para la construcción de Servicios Web y aplicaciones Java REST. Representa una implementación completa y portable de la especificación JAX-RS a través del protocolo HTTP. Se puede ejecutar en cualquier contenedor de Serles, pero proporciona una mayor integración con JBoss Application Server.

Regla: Para implementaciones de proveedores, mapeos y transformaciones de datos con formato JSON se debe considerar como primera instancia el uso del API Jackson.

Utilidad: Jackson es una biblioteca de Java de usos múltiples para procesar formato de datos JSON y XML disponible para la plataforma StAX, JAXB, etc. Se integra con algunas implementaciones de JAX-RS (Jersey, Apache CXF, RESTEasy, Restlet) y el marco de trabajo de Spring.

Regla: Para la serialización y deserialización entre objetos Java y su representación en notación JSON se debe usar Jackson o el API Google Gson, de preferencia Jackson.

6.2.2 Implementación

Regla: Un URI REST debe referirse a un recurso como una entidad de negocio y no como una acción. Los nombres de recursos deben ser sustantivos pluralizados y con estilo: “lowerCamelCase”.

Ejemplo:

```
/* Servicio para administrar la información de los sujetos del módulo de Catastro */
```

```
https://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos
```

```
/* Servicio para administrar la información de las unidades administrativas del módulo de ADM */
```

```
https://www.sri.gob.ec/sri-administracion-interfaz/rest/unidadesAdministrativas
```

Regla: Los URI deben seguir una estructura jerárquica predecible para mejorar la comprensibilidad y, por lo tanto, la usabilidad. Predecibles en el sentido de que son coherentes y jerárquicos en el sentido de que los datos tienen relaciones de estructura.

```
/* Inserta un nuevo sujeto en el módulo de Catastro */
```

```
POST https://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos
```

```
/* Obtiene un sujeto del módulo de Catastro con ID: 33457 */
```

```
GET http://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457
```

```
/* Actualizar un sujeto del módulo de Catastro con ID: 33457 */
```

```
PUT https://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457
```

```
/* Elimina un sujeto del módulo de Catastro con ID: 33457 */
```

```
DELETE https://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457
```

```
/* Obtiene los todos los medios de contacto del sujeto de Catastro con ID: 33457 */  
GET https://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457/mediosContacto
```

Regla: Cuando no exista un concepto de colección o entidad de negocio es aceptable usar un nombre de recurso en singular.

```
/* Servicio que obtiene una configuración global desde el módulo de Administración */  
GET https://www.sri.gob.ec/sri-administracion-interfaz/rest/configuracion
```

Utilidad: Nombres de recursos apropiados proporcionan contexto para una solicitud de servicio, aumentando la comprensión de la API de servicio. Los recursos se ven jerárquicamente a través de sus nombres de URI, ofreciendo a los consumidores una jerarquía amigable y fácil de entender para aprovechar al máximo sus aplicaciones. Cuando los recursos se nombran bien, una API es intuitiva y fácil de usar.

Consideraciones:

- Las API REST están escritas para los consumidores. El nombre y la estructura de los URIs deben transmitir significado. Diseñe para sus clientes, no para sus datos.
- Los recursos que se ven jerárquicamente a través de los URIs es un análisis sobre la granularidad del servicio, contexto de negocio y esquema relacional.

Regla: El uso de URIs configuradas con parámetros basados en expresiones regulares deben contar con la aprobación del Arquitecto TI asignado. Para los casos donde esté justificado en su uso estos no pueden figurar al inicio de la estructura de la URI, sino que deben anteponer siempre un segmento de URL estático.

Ejemplo:

```
//Mal  
@GET  
@Path("{orderId:[a-z]\\d{2,3}}")  
public String getOrders2(@PathParam("orderId") String orderId) {  
    return "orderId: " + orderId;  
}  
  
//Bien  
@GET  
@Path("obtenerOrden/{orderId:[a-z]\\d{2,3}}")  
public String getOrders2(@PathParam("orderId") String orderId) {  
    return "orderId: " + orderId;  
}
```

Regla: Los servicios REST deben implementarse con sus respectivos métodos de petición HTTP, alineándose a los principios de idempotencia según las siguientes directrices:

Método HTTP	Idempotente	Lineamiento
GET	Si	Utilizar para capacidades de lectura y obtención de datos. Este método no puede cambiar datos. No exponga operaciones inseguras a través de este método.
PUT	Si	Utilizar para capacidades de actualización.
DELETE	Si	Utilizar para capacidades de eliminación. Un borrado lógico, aunque representa una actualización de estados deberá desarrollarse con el verbo: "DELETE" ya que realiza una eliminación de recursos identificado por un URI.
POST	No	Utilizar para capacidades que crean nuevos recursos.
HEAD	Si	Utilizar este método para lectura y obtención de metadatos de los encabezados de respuesta. No exponga operaciones inseguras a través de este método.
OPTION	Si	Utilizar este método para comprobar la funcionalidad de un servidor web mediante petición en lugar de un recurso específico. No exponga operaciones inseguras a través de este método.
TRACE	Si	Utilizar este método para fines de depuración y diagnóstico. No exponga operaciones inseguras a través de este método.

Regla: Los servicios web que tienen el propósito de guardar o actualizar información solo pueden enviar los datos en el body del request y no directamente en la URL.

6.2.3 Validaciones con JavaBeans

Regla: Los parámetros de los métodos que se implementan como Servicios REST y que requieran de validación o restricciones (constraints) debe usar el API de Bean Validation con su respectivo archivo de propiedades para la personalización de mensajes.

```
/**
 * Obtiene todos los sujetos del módulo de catastro por estado
 */
@GET
public Response obtenerPorEstado(@NotNull(message = "{sujeto.consulta.estado.requerido}")
@QueryParam("estado") String estado) {
```

Utilidad: JAX-RS soporta Bean Validation para verificar los parámetros utilizados en los recursos REST y su uso permite tener un esquema único de validación que se propaga por todas las capas de la aplicación.

Verifique también [Validación de servicios web REST](#) y [Mapeo de excepciones de Bean Validation con JAX-RS](#).

6.2.4 Respuestas

Regla: Los servicios REST deben retornar un objeto `jakarta.ws.rs.core.Response`, el mismo que deberá tener incluidos los datos que se desea retornar si los hubiera y el código de retorno acorde a la tabla de códigos de estado respuesta.

Utilidad: El objeto `jakarta.ws.rs.core.Response` permite agregar metadatos como cabeceras y códigos de respuesta.

Ejemplo:

```
@GET
@Path("/{messageId}")
@Produces(MediaType.APPLICATION_JSON)
public Response getMessage(@PathParam("messageId") long id) {

    Map<String, String> map = new HashMap<>();

    map.put("1", "abc");
    map.put("2", "def");
    map.put("3", "ghi");

    return Response.status(Status.OK).entity(map).build();}
```

Output: { "1": "abc", "2": "def", "3": "ghi" }

Regla: Los códigos de estado que se debe implementar en las respuestas de los servicios REST son:

Código de Estado	Descripción	Contexto de Implementación
200	OK	Utilizar para emitir un resultado de éxito general con contenido en el cuerpo de la respuesta.
201	Creado	Utilizar para emitir un resultado de éxito por una creación exitosa.
204	OK Sin Contenido	Utilizar para emitir un resultado de éxito general sin contenido en el cuerpo de la respuesta.
400	Petición Incorrecta	Utilizar para emitir un resultado de error por validaciones o restricciones en los parámetros de la petición.
401	No Autorizado	Utilizar para emitir un resultado de error de la petición por que el cliente no está autenticado y el recurso es privado.
403	Prohibido	Utilizar para emitir un resultado de error de la petición por que el cliente no está autorizado para realizar la operación solicitada
404	No Encontrado	Utilizar para emitir un resultado de error de la petición por ser un recurso no establecido o inexistente
500	Error Interno del Servidor	Utilizar para emitir un resultado de error general cuando el servidor lanza una excepción no controlada.
503	Servicio Indisponible	Utilizar para emitir un resultado de error cuando el servicio está congestionado o por tareas de mantenimiento

7 Glosario

7.1 Acrónimos

- **API** Application Programming Interface
- **CDI** Contexts and Dependency Injection
- **CMT** Container-Managed Transactions
- **DAO** Data Access Object
- **DDL** Data Definition Language
- **DML** Data Manipulation Language
- **DTO** Data Transfer Object
- **EJB** Enterprise JavaBeans (Enterprise Beans)
- **EL** Expression Language
- **HTTPS** Hypertext Transfer Protocol Secure
- **JAX-RS** Jakarta RESTful Web Services
- **JAX-WS** Jakarta XML Web Services
- **JMS** Jakarta Message Service
- **JPA** Jakarta Persistence API
- **JPQL** Jakarta Persistence Query Language
- **JSON** JavaScript Object Notation
- **JSF** JakartaServer Faces
- **JTA** Jakarta Transaction API
- **JVM** Java Virtual Machine
- **MDC** Mapped Diagnostic Context
- **MDB** Message-Driven Bean
- **ORM** Object-Relational Mapping
- **POJO** Plain Old Java Object
- **REST** Representational State Transfer
- **SOAP** Simple Object Access Protocol
- **TDD** Test-Driven Development
- **URI** Uniform Resource Identifier
- **URL** Uniform Resource Locator
- **UUID** Universally Unique Identifier
- **WSDL** Web Services Description Language
- **XML** eXtensible Markup Language
- **XSD** XML Schema Definition

7.2 Definiciones

- **API:** Conjunto de interfaces, clases y contratos que permiten utilizar las funcionalidades proporcionadas por una tecnología o plataforma.
- **Aplicación Empresarial:** Sistema de software desarrollado sobre la plataforma Jakarta EE que implementa procesos de negocio institucionales.
- **Clase Embeddable:** Clase anotada con `@Embeddable` que representa un conjunto reutilizable de atributos persistentes que forman parte de una entidad y no poseen identidad propia.

- **Componente Administrado (Managed Bean):** Clase cuyo ciclo de vida es administrado por el contenedor Jakarta EE y que puede beneficiarse de servicios como inyección de dependencias, gestión transaccional, interceptores, eventos y validación.
- **Contenedor Jakarta EE:** Infraestructura de ejecución responsable de administrar el ciclo de vida de los componentes, la seguridad, las transacciones, la concurrencia, la inyección de dependencias y demás servicios definidos por las especificaciones Jakarta EE.
- **Contexto de Diagnóstico Mapeado (MDC):** Mecanismo que permite asociar información contextual (por ejemplo, identificador de correlación, usuario o transacción) a los registros de log generados durante la ejecución de una solicitud.
- **Contexto Transaccional:** Entorno administrado por Jakarta Transactions que define el alcance y comportamiento de una transacción durante la ejecución de una operación de negocio.
- **Contexto de Persistencia:** Conjunto de entidades administradas por un EntityManager, encargado de sincronizar el estado de los objetos con la base de datos.
- **Disposer:** Método anotado con @Disposes utilizado para liberar o finalizar correctamente recursos creados mediante un Producer.
- **Excepción de Aplicación:** Excepción que representa una condición funcional o una regla de negocio esperada durante la ejecución de la aplicación.
- **Excepción del Sistema:** Excepción que representa un error técnico o de infraestructura no previsto, como problemas de conectividad, persistencia, recursos o fallos de ejecución.
- **Inyección de Dependencias:** Mecanismo mediante el cual el contenedor suministra automáticamente las dependencias requeridas por un componente.
- **Logging:** Mecanismo para registrar eventos relevantes producidos durante la ejecución de una aplicación con fines de auditoría, diagnóstico, monitoreo y soporte operativo.
- **Productor (Producer):** Método o campo anotado con @Produces utilizado para crear y suministrar objetos administrados por CDI cuando su construcción requiere lógica personalizada.
- **Prueba Unitaria:** Prueba automatizada que verifica el comportamiento de una unidad de software de manera aislada, utilizando objetos simulados (mocks) cuando sea necesario.
- **Servicio de Negocio:** Componente que implementa la lógica de negocio de una aplicación. En Jakarta EE podrá implementarse mediante servicios CDI o Enterprise Beans (EJB).
- **Servicio CDI:** Componente administrado por CDI utilizado para implementar lógica de negocio mediante inyección de dependencias y, cuando corresponda, gestión transaccional mediante @Transactional.
- **Transacción:** Unidad lógica de trabajo compuesta por una o más operaciones que deben ejecutarse de forma atómica, consistente, aislada y duradera (ACID).
- **TDD (Test-Driven Development):** Metodología de desarrollo en la cual las pruebas unitarias se implementan antes del código funcional, siguiendo el ciclo Red – Green – Refactor.

8 Bibliografía

1. Jakarta EE. (2026). Jakarta EE Platform: <https://jakarta.ee/specifications/platform/>
2. Tutorial oficial de Jakarta EE. (2026). <https://jakarta.ee/learn/docs/jakartaee-tutorial/current/>
3. Chang, H., Mariani, L., and Pezz'e, M. 2013. Exception handlers for healing component-based systems. ACM Trans. Softw. Eng. Methodol. 22, 4, Article 30 (October 2013), 40 pages.
4. Gang Huang, Yihan Wu. Towards Architecture-Level Middleware-Enabled Exception Handling of Component-based Systems. The 14th International ACM SIGSOFT Symposium on Components Based Software Engineering (CBSE-2011), Jun 2011, Boulder, Colorado, United States.
5. Gregor Hohpe (2014), The Enterprise Patterns. Consultado de la página web <http://www.eaipatterns.com/eaipatterns.html>, obtenido en marzo del 2015.
6. Gregor Hohpe, Bobby Woolf. 2004 Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley ISBN 0321200683, páginas 57 a la 97.
7. IMB (2015), Principles of XML design: When to use elements versus attributes, <http://www.ibm.com/developerworks/library/x-eleatt/index.html>
8. Oracle. (2014). The Java EE 7 Tutorial. Consultado de la página web <https://docs.oracle.com/javaee/7/tutorial/>, obtenido en marzo del 2015.
9. PMD. (2015). Consultado de la página web <http://pmd.sourceforge.net/>, obtenido el 07 de Marzo del 2015.
10. The Java API for XML-Based Web Services (JAX-WS) 2.2 Rev a (Maintenance Release) May 13, 2011

9 Anexos

9.1 Anexo 1: Estándares soportadas por la Plataforma Jakarta EE

Hasta la fecha de actualización de este documento, los siguientes estándares de Jakarta EE (anteriormente Java™ EE) son compatibles con JBoss EAP 8.1.⁶

Estándar	Versión	Estándar	Versión
Jakarta EE (anteriormente Java EE)	10	Jakarta Standard Tag Library (anteriormente Standard Tag Library for JavaServer Pages) (JSTL)	3.0
Java EE	--	Jakarta Annotations (anteriormente Common Annotations for Java)	2.1
Jakarta XML RPC (anteriormente Java API for XML-based RPC) (JAX-RPC)	--	Jakarta XML Binding (anteriormente Java Architecture for XML Binding) (JAXB)	4.0
Jakarta SOAP with Attachments (anteriormente SOAP with Attachments API for Java) (SAAJ)	3.0	Jakarta RESTful Web Services (anteriormente Java API for RESTful Web Services) (JAX-RS)	3.1
Jakarta Servlet (anteriormente Java Servlet)	6.0	Jakarta XML Registries (anteriormente Java API for XML Registries) (JAXR)	--
Jakarta Server Faces (anteriormente Java Server Faces) (JSF)	4.0	Jakarta Contexts and Dependency Injection (anteriormente Contexts and Dependency Injection for the Java EE) (CDI)	4.0
Jakarta Server Pages (anteriormente Java Server Pages) (JSP)	3.1	Jakarta Bean Validation (anteriormente Bean Validation)	3.0
Jakarta Transactions (anteriormente Java Transaction API) (JTA)	2.0	Jakarta JSON Processing (anteriormente Java API for JSON Processing)	2.1
Jakarta Authorization (anteriormente Java Authorization Service Provider Contract for Containers) (JAAC)	2.1	Jakarta Security (anteriormente Java EE Security API)	3.0

⁶ <https://access.redhat.com/articles/113373>

Jakarta Enterprise Beans (anteriormente Enterprise Java Beans) (EJB)	4.0	Jakarta JSON Binding (anteriormente Java API for JSON Binding)	3.0
Jakarta Connectors (anteriormente Java EE Connector Architecture) (JCA)	2.1	Jakarta Concurrency (anteriormente Concurrency Utilities for Java EE)	3.0
Jakarta Management (anteriormente Java Platform, Enterprise Edition Management)	--	Jakarta Dependency Injection	2.0
Jakarta Enterprise Web Services (anteriormente Web Services for Java EE)	2.0	Jakarta Expression Language	5.0
Jakarta Deployment (anteriormente Java Application Deployment)	--	Jakarta Interceptors	2.1
Jakarta Mail (anteriormente JavaMail API)	2.1	Jakarta WebSocket	2.1
Jakarta Messaging (anteriormente Java Message Services) (JMS)	3.1	Jakarta Activation	2.1
Jakarta Persistence (anteriormente Java Persistence API) (JPA)	3.1	Jakarta Authentication	3.0
Jakarta Web Services Metadata (anteriormente Web Services Metadata for the Java Platform)	-- ⁷	Jakarta Batch	2.1
Jakarta Web Services (anteriormente Web Services for Java) (JAX-WS)	4.0		
Jakarta Debugging Support for Other Languages (anteriormente Debugging Support for Other Languages)	2.0		

⁷ Jakarta Web Services Metadata se fusionaron con la Jakarta XML Web Services Specification (JAXWS)

9.2 Anexo 2: Tecnologías/Herramientas a utilizar

Tecnología	Versión
OpenJDK	21.0.12 o superior
Apache Maven	3.9.16 o superior
SOAP	1.2
ELK Stack, Logstash: (visualización y análisis de Logs)	https://srikbn02.sri.ad/s/monitoreo_logs/app/discover/

9.3 Anexo 3: Recursos relacionados

Los documentos o recursos mencionados en este documento con sus respectivas ubicaciones se detallan a continuación:

Documento/Recurso	Ubicación (Alfresco / Git)
Proyecto base para aplicaciones con Plataforma Jakarta EE.	Importante: Leer archivo readme.md del proyecto base correspondiente a utilizar. Base Intranet: https://gitlab.sri.ad/Arquetipos/sri-base Base Internet: https://gitlab.sri.ad/Arquetipos/sri-base-internet Base Batch: https://gitlab.sri.ad/Arquetipos/batch/sri-base-batch Base Util: https://gitlab.sri.ad/Arquetipos/sri-base-util/ Repositorio Nexus: http://sridesa02:8081/
Reglas definidas de PMD	https://gitlab.sri.ad/Arquetipos/reglas-codigoestatico-java/sri-reglas-pmd/-/blob/main/sri-reglas-pmd.md
Reglas definidas de Checkstyle	https://gitlab.sri.ad/Arquetipos/reglas-codigoestatico-java/sri-reglas-checkstyle/-/blob/main/sri-reglas-checkstyle.md
Reglas definidas de SpotBugs	https://gitlab.sri.ad/Arquetipos/reglas-codigoestatico-java/sri-reglas-spotbugs/-/blob/main/sri-reglas-spotbugs.md
Estándar de Desarrollo – Objetos de Base de Datos vigente	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO BASE DE DATOS> VIGENTES
Estándar de Desarrollo Aplicaciones - Estructura y Nomenclatura vigente	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO JEE7> VIGENTES
Estándar Tecnológico Codificación Lenguaje de Programación Java SE vigente	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA

	TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO PLATAFORMA JAKARTA EE> VIGENTE
Arquitectura Tecnológica De Referencia General vigente	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 2. ARQUITECTURA TECNOLÓGICA DE APLICACIÓN, DATOS, INFRAESTRUCTURA> ARQUITECTURAS DE REFERENCIA> VIGENTE
Arquitectura Tecnológica de Referencia de Software vigente	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 2. ARQUITECTURA TECNOLÓGICA DE APLICACIÓN, DATOS, INFRAESTRUCTURA> ARQUITECTURAS DE REFERENCIA> VIGENTE
Guía Referencial de Patrones de Diseño Empresariales – Plataforma Jakarta EE	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO PLATAFORMA JAKARTA EE> VIGENTE
Estándares de Desarrollo de Aplicaciones Batch	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO JEE7> VIGENTES
Estándar de Configuración de Servidores de Aplicaciones JBoss EAP	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO PLATAFORMA JAKARTA EE> VIGENTE
Guía de Implementación de Pistas de Auditoría	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO SEGURIDAD> VIGENTES
Estándar Tecnológico Seguridad deAPI Web	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> DESARROLLO SEGURIDAD> VIGENTES
RESOLUCION N° 044-NG-DINARDAP-2016 ANEXO_1	Repositorio> SRI> Coordinación de Inteligencia de la Información> Documentos Inteligencia de la Informacion> Base Legal y Resoluciones de DINARDAP
Guía Administración Hat Single Sign-On	Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 3. ESTÁNDARES Y GUÍAS DE IMPLEMENTACIÓN> REDHAT SINGLE SIGN ON> VIGENTES